



Republic of Tunisia  
Ministry of Higher Education  
and Scientific Research



**IIT**  
INSTITUT  
INTERNATIONAL  
TECHNOLOGIE

Private International School of  
Technology at Sfax / IIT

---

Al-Madrassa al-'Ulya al-Dawliyya al-Khassa lil-Tiknulujjiya bi-Sfax

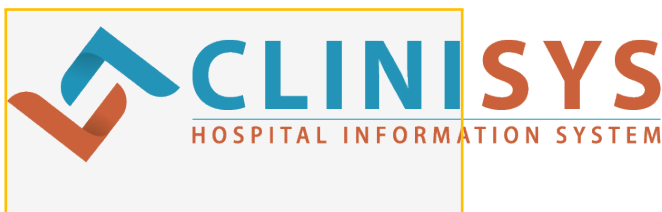
**COMPUTER SCIENCES DEPARTMENT**

---

# GRADUATION PROJECT

In view of obtaining the **National Diploma of 3 years bachelor**  
in **COMPUTER SCIENCES**

**Performed at :**



Elaborated by :

**Youssef AYADI**

---

Entitled :

## **Smart Internship and Project Management System — SIPMS**

Defended on : \_\_\_\_\_ / \_\_\_\_\_ / 2026

**Committee :**

Mr. / Mrs. \_\_\_\_\_ – Chairman  
Mr. / Mrs. \_\_\_\_\_ – Examiner  
Mr. / Mrs. \_\_\_\_\_ – Academic supervisor  
Mr. / Mrs. \_\_\_\_\_ – Industrial supervisor

---

**ACADEMIC YEAR : 2025 – 2026**



*To my beloved parents,  
whose unconditional love, endless sacrifices,  
and unwavering belief in me have been  
the greatest source of strength throughout my journey.*

*To my family and friends,  
for their constant encouragement and support.*

*To all my teachers and mentors  
who have guided me with knowledge and wisdom.*

*This work is dedicated to you all.*

**Youssef Ayadi**



---

# ACKNOWLEDGEMENTS

First and foremost, I would like to express my deepest gratitude to **Allah** for granting me the strength, patience, and perseverance to complete this work.

I would like to express my deepest gratitude to my academic supervisor, **Mrs. Rahma Bouaziz**, for her invaluable guidance, constructive feedback, and continuous support. I am also profoundly thankful to my industrial supervisor, **Mr. Marwen Hadrich**, for his mentorship, expertise, and for welcoming me into **CLINISYS**.

I would also like to extend my sincere thanks to the Committee President, **Mr. Fahmi Kallel**, and to the honorable Reviewer for accepting to evaluate this work. Their insights and expertise are highly appreciated.

I extend my sincere thanks to the entire team at **CLINISYS** for welcoming me into their organization and providing an enriching and stimulating professional environment. Their collaboration and trust were essential to the success of this project.

My heartfelt appreciation goes to the faculty and staff of **IIT Sfax** for the quality education and technical training they have provided throughout my studies. The knowledge and skills acquired during my academic journey have been the foundation upon which this project was built.

I am also deeply grateful to my family, whose love, encouragement, and sacrifices have sustained me throughout my academic career. A special thank you to my friends and colleagues for their moral support and camaraderie.

Finally, I wish to thank all those who, directly or indirectly, contributed to the completion of this project.

**Youssef Ayadi**



---

# TABLE OF CONTENTS

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>Acknowledgements</b>                             | <b>2</b>  |
| <b>LIST OF FIGURES</b>                              | <b>8</b>  |
| <b>List of Abbreviations</b>                        | <b>9</b>  |
| <b>GENERAL INTRODUCTION</b>                         | <b>10</b> |
| <b>1 GENERAL FRAMEWORK</b>                          | <b>12</b> |
| 1.1 INTRODUCTION . . . . .                          | 13        |
| 1.2 PRESENTATION OF THE HOST ORGANIZATION . . . . . | 13        |
| 1.2.1 About CLINISYS . . . . .                      | 13        |
| 1.2.2 Key Activity Areas . . . . .                  | 13        |
| 1.3 ANALYSIS OF THE EXISTING SYSTEM . . . . .       | 14        |
| 1.3.1 Description of the Current System . . . . .   | 14        |
| 1.3.2 Identified Limitations . . . . .              | 14        |
| 1.4 PROBLEM STATEMENT . . . . .                     | 14        |
| 1.5 PROPOSED SOLUTION . . . . .                     | 15        |
| 1.5.1 The SIPMS Platform . . . . .                  | 15        |
| 1.5.2 Technology Stack Overview . . . . .           | 16        |
| 1.6 DEVELOPMENT METHODOLOGY . . . . .               | 16        |
| 1.6.1 Choice of Methodology : Scrum . . . . .       | 16        |
| 1.6.2 Scrum Framework . . . . .                     | 16        |
| 1.6.3 Product Backlog . . . . .                     | 17        |
| 1.6.4 Sprint Goals . . . . .                        | 20        |
| 1.6.5 Validation per Sprint . . . . .               | 20        |
| 1.6.6 Feedback Loop . . . . .                       | 21        |
| 1.7 CONCLUSION . . . . .                            | 22        |
| <b>2 REQUIREMENTS ANALYSIS AND SPECIFICATION</b>    | <b>23</b> |

|          |                                              |           |
|----------|----------------------------------------------|-----------|
| 2.1      | INTRODUCTION . . . . .                       | 24        |
| 2.2      | SYSTEM ACTORS . . . . .                      | 24        |
| 2.3      | FUNCTIONAL REQUIREMENTS . . . . .            | 24        |
| 2.3.1    | Authentication and Access Control . . . . .  | 24        |
| 2.3.2    | Reception and Candidate Management . . . . . | 25        |
| 2.3.3    | Quiz and Evaluation . . . . .                | 25        |
| 2.3.4    | Project Management . . . . .                 | 25        |
| 2.3.5    | Application Lifecycle . . . . .              | 26        |
| 2.3.6    | Interview Management . . . . .               | 26        |
| 2.3.7    | Application Lifecycle . . . . .              | 26        |
| 2.3.7.1  | State Machine Algorithm . . . . .            | 26        |
| 2.3.7.2  | Validation Metrics . . . . .                 | 27        |
| 2.3.7.3  | Benchmark Results . . . . .                  | 27        |
| 2.3.8    | AI Module . . . . .                          | 28        |
| 2.4      | NON-FUNCTIONAL REQUIREMENTS . . . . .        | 29        |
| 2.5      | SYSTEM DECOMPOSITION . . . . .               | 29        |
| 2.6      | USE CASE MODELING . . . . .                  | 31        |
| 2.7      | CONCLUSION . . . . .                         | 35        |
| <b>3</b> | <b>SYSTEM DESIGN</b>                         | <b>37</b> |
| 3.1      | INTRODUCTION . . . . .                       | 38        |
| 3.2      | GENERAL ARCHITECTURE . . . . .               | 38        |
| 3.2.1    | Architectural Pattern . . . . .              | 38        |
| 3.2.2    | Backend Package Structure . . . . .          | 39        |
| 3.2.3    | Security Architecture . . . . .              | 40        |
| 3.3      | CLASS DIAGRAM . . . . .                      | 40        |
| 3.3.1    | Core Entity Relationships . . . . .          | 40        |
| 3.3.1.1  | Core Domain Subsystem . . . . .              | 41        |
| 3.3.1.2  | Quiz Subsystem . . . . .                     | 43        |
| 3.3.1.3  | AI Subsystem . . . . .                       | 43        |
| 3.3.2    | Relationship Diagram . . . . .               | 43        |
| 3.3.3    | Entity Interaction and Call Flow . . . . .   | 46        |
| 3.3.4    | AI Entities and Scoring Logic . . . . .      | 47        |
| 3.3.4.1  | AI Scoring Fields . . . . .                  | 47        |
| 3.3.4.2  | AiRanking Entity . . . . .                   | 47        |
| 3.3.4.3  | AI Scoring Algorithm . . . . .               | 48        |

|          |                                                                         |           |
|----------|-------------------------------------------------------------------------|-----------|
| 3.4      | SEQUENCE DIAGRAMS . . . . .                                             | 48        |
| 3.4.1    | Sequence : Candidate Registration with Internship File Upload . . . . . | 49        |
| 3.4.2    | Sequence : Approve Candidate and Send Quiz . . . . .                    | 50        |
| 3.4.3    | Sequence : Interview Scheduling and Result Recording . . . . .          | 51        |
| 3.5      | DATABASE SCHEMA . . . . .                                               | 54        |
| 3.5.1    | Main Tables . . . . .                                                   | 54        |
| 3.5.2    | Normalization Analysis . . . . .                                        | 55        |
| 3.5.3    | Constraints and Integrity Rules . . . . .                               | 55        |
| 3.5.4    | Schema Design Decisions . . . . .                                       | 56        |
| 3.5.5    | Diagram Quality and Formal Coherence . . . . .                          | 57        |
| 3.6      | CONCLUSION . . . . .                                                    | 58        |
| <b>4</b> | <b>IMPLEMENTATION</b>                                                   | <b>59</b> |
| 4.1      | INTRODUCTION . . . . .                                                  | 60        |
| 4.2      | DEVELOPMENT ENVIRONMENT . . . . .                                       | 60        |
| 4.2.1    | Hardware Environment . . . . .                                          | 60        |
| 4.2.2    | Software Environment . . . . .                                          | 60        |
| 4.3      | SPRINT 1 — AUTHENTICATION AND USER MANAGEMENT . . . . .                 | 61        |
| 4.3.1    | Backend Components Built . . . . .                                      | 61        |
| 4.3.2    | Frontend Components Built . . . . .                                     | 61        |
| 4.3.3    | Delivered Interfaces . . . . .                                          | 62        |
| 4.4      | SPRINT 2 — RECEPTION MODULE . . . . .                                   | 63        |
| 4.4.1    | Backend Components Built . . . . .                                      | 63        |
| 4.4.2    | Frontend Components Built . . . . .                                     | 63        |
| 4.4.3    | Delivered Interfaces . . . . .                                          | 64        |
| 4.5      | SPRINT 3 — QUIZ MODULE . . . . .                                        | 65        |
| 4.5.1    | Backend Components Built . . . . .                                      | 65        |
| 4.5.2    | Frontend Components Built . . . . .                                     | 65        |
| 4.5.3    | Delivered Interfaces . . . . .                                          | 66        |
| 4.6      | SPRINT 4 — INTERVIEW, AI, AND ASSIGNMENT MODULE . . . . .               | 66        |
| 4.6.1    | Backend Components Built . . . . .                                      | 66        |
| 4.6.2    | Frontend Components Built . . . . .                                     | 67        |
| 4.6.3    | Delivered Interfaces . . . . .                                          | 68        |
| 4.7      | EVALUATION . . . . .                                                    | 69        |
| 4.7.1    | Code Quality . . . . .                                                  | 69        |
| 4.7.2    | API Performance . . . . .                                               | 69        |



## TABLE OF CONTENTS

---

|          |                                               |           |
|----------|-----------------------------------------------|-----------|
| 4.7.3    | Security Assessment . . . . .                 | 70        |
| 4.7.4    | Coverage Summary . . . . .                    | 70        |
| 4.8      | CONCLUSION . . . . .                          | 70        |
| <b>5</b> | <b>TESTING AND VALIDATION</b>                 | <b>72</b> |
| 5.1      | INTRODUCTION . . . . .                        | 73        |
| 5.2      | TESTING STRATEGY . . . . .                    | 73        |
| 5.2.1    | Unit Testing . . . . .                        | 73        |
| 5.2.2    | Integration Testing . . . . .                 | 74        |
| 5.2.3    | System Testing . . . . .                      | 74        |
| 5.2.4    | User Acceptance Testing (UAT) . . . . .       | 74        |
| 5.3      | TEST CASES . . . . .                          | 76        |
| 5.4      | PERFORMANCE TESTING . . . . .                 | 77        |
| 5.4.1    | Response Time Validation . . . . .            | 77        |
| 5.4.2    | Load Handling . . . . .                       | 77        |
| 5.5      | SECURITY TESTING . . . . .                    | 77        |
| 5.5.1    | Role-Based Access Control (RBAC) . . . . .    | 78        |
| 5.5.2    | JWT Validation . . . . .                      | 78        |
| 5.6      | VALIDATION RESULTS . . . . .                  | 78        |
| 5.7      | CONCLUSION . . . . .                          | 80        |
| <b>6</b> | <b>DISCUSSION AND EVALUATION</b>              | <b>81</b> |
| 6.1      | INTRODUCTION . . . . .                        | 82        |
| 6.2      | EVALUATION OF OBJECTIVES . . . . .            | 82        |
| 6.3      | TECHNOLOGY EVALUATION . . . . .               | 83        |
| 6.3.1    | Backend : Spring Boot 3.2 + Java 17 . . . . . | 83        |
| 6.3.2    | Frontend : React 19 + Vite . . . . .          | 83        |
| 6.3.3    | Database : MySQL 8 . . . . .                  | 83        |
| 6.3.4    | AI Integration . . . . .                      | 83        |
| 6.4      | LIMITATIONS . . . . .                         | 83        |
| 6.5      | FUTURE ENHANCEMENTS . . . . .                 | 84        |
| 6.6      | CONCLUSION . . . . .                          | 84        |
|          | <b>GENERAL CONCLUSION</b>                     | <b>85</b> |
|          | <b>ANNEXES</b>                                | <b>89</b> |
| A.1      | TITRE ANNEXE 1 . . . . .                      | 89        |



---

# LIST OF FIGURES

|      |                                                                                   |    |
|------|-----------------------------------------------------------------------------------|----|
| 2.1  | Global use case diagram — SIPMS platform . . . . .                                | 31 |
| 3.1  | Three-tier architecture of the SIPMS platform . . . . .                           | 39 |
| 3.2  | Simplified class diagram — SIPMS core entities . . . . .                          | 45 |
| 3.3  | Sequence diagram — Candidate registration with document upload . . . . .          | 50 |
| 4.1  | Login page — SIPMS CLINISYS branding . . . . .                                    | 62 |
| 4.2  | Administrator dashboard — Users management panel . . . . .                        | 62 |
| 4.3  | Reception Panel — Candidate list with year filter . . . . .                       | 64 |
| 4.4  | New Candidate registration modal with document upload . . . . .                   | 64 |
| 4.5  | Internship Files tab — All files across candidates . . . . .                      | 64 |
| 4.6  | Quiz interface — Question view with countdown timer . . . . .                     | 66 |
| 4.7  | Quiz selection modal — Manager chooses existing quiz or creates default . . . . . | 66 |
| 4.8  | Interview management page — Schedule and record results . . . . .                 | 68 |
| 4.9  | AI Insights page — Project rankings and candidate matching . . . . .              | 68 |
| 4.10 | CV Upload page — AI analysis, project matching, and roadmap . . . . .             | 68 |
| A.1  | Titre de figure (exemple) . . . . .                                               | 89 |



---

# LIST OF ABBREVIATIONS

| Abbreviation | Definition                                     |
|--------------|------------------------------------------------|
| AI           | Artificial Intelligence                        |
| API          | Application Programming Interface              |
| RBAC         | Role-Based Access Control                      |
| JWT          | JSON Web Token                                 |
| REST         | Representational State Transfer                |
| HTTP         | HyperText Transfer Protocol                    |
| HTTPS        | HyperText Transfer Protocol Secure             |
| JPA          | Java Persistence API                           |
| ORM          | Object-Relational Mapping                      |
| MVC          | Model-View-Controller                          |
| SPA          | Single-Page Application                        |
| SQL          | Structured Query Language                      |
| SMTP         | Simple Mail Transfer Protocol                  |
| CSS          | Cascading Style Sheets                         |
| HTML         | HyperText Markup Language                      |
| UI           | User Interface                                 |
| UX           | User Experience                                |
| SIPMS        | Smart Internship and Project Management System |
| MCQ          | Multiple-Choice Question                       |
| CIN          | National Identity Card Number                  |
| CV           | Curriculum Vitae                               |
| PFE          | Final Year Project (Projet de Fin d'Études)    |
| IDE          | Integrated Development Environment             |
| BCrypt       | Blowfish Crypt (password hashing algorithm)    |
| KPI          | Key Performance Indicator                      |

**Abbreviations used in this report**



---

# GENERAL INTRODUCTION

In today's digitally driven world, organizations are under constant pressure to streamline their internal processes and improve operational efficiency. Among the administrative challenges faced by technology companies, the management of internship candidates stands out as a particularly complex and resource-intensive task. When handled manually, it involves paper-based registration, unstructured document storage, time-consuming evaluations, and error-prone communication workflows.

This project was initiated in response to the growing administrative burden faced by technology companies in managing internship candidacies manually. Carried out at **CLINISYS**, a company specializing in advanced technological solutions, this project aims to design and develop the **Smart Internship and Project Management System (SIPMS)** — a full-stack web platform designed to digitize and automate the complete lifecycle of internship candidates, from registration to project assignment.

The SIPMS platform is built on a modern, robust technology stack : a reactive frontend powered by **React 19 (Vite 8)**, a secure backend built with **Spring Boot 3.2 (Java 17)**, a relational database using **MySQL 8**, and a stateless authentication system based on **JSON Web Tokens (JWT)**. It further incorporates artificial intelligence features for project ranking, candidate-to-supervisor matching, CV text analysis, and roadmap generation, alongside project management, application lifecycle tracking with a deterministic state machine, interview scheduling, automated email notifications, audit logging, and multi-role dashboards.

This report is organized into six main chapters :

- **Chapter 1 — General Framework** : Presentation of the host organization, analysis of the existing system, problem statement, proposed solution, and development methodology.

- **Chapter 2 — Requirements Analysis and Specification :** Identification of actors, functional and non-functional requirements, and use case modeling.
- **Chapter 3 — System Design :** Overall architecture, class diagrams, sequence diagrams, and database schema.
- **Chapter 4 — Implementation :** Development environment, Scrum sprint execution, and presentation of the realized user interfaces.
- **Chapter 5 — Testing and Validation :** Testing strategy, test cases, performance and security testing, validation results.
- **Chapter 6 — Discussion and Evaluation :** Evaluation of objectives, technology assessment, limitations, and future enhancements.

We conclude this report with a summary of the achieved results and future enhancement perspectives for the SIPMS platform.

---

# **GENERAL FRAMEWORK**

# 1.1 INTRODUCTION

This chapter presents the general framework of our final-year project. We begin by introducing the host organization CLINISYS, followed by a critical analysis of the existing system. We then define the core problem statement, present the proposed SIPMS solution, and describe the Scrum agile methodology adopted for the development process.

# 1.2 PRESENTATION OF THE HOST ORGANIZATION

## 1.2.1 About CLINISYS

CLINISYS is a technology company specializing in the development of innovative digital solutions for industrial and academic sectors. With deep expertise in software engineering, the company supports its clients in their digital transformation journeys by delivering custom-built systems that combine performance, security, and usability.

As part of its commitment to talent development and innovation, CLINISYS welcomes a growing number of interns from higher education institutions each year. This ongoing engagement with academic talent has highlighted the urgent need for a structured digital tool capable of managing the full internship lifecycle efficiently.

## 1.2.2 Key Activity Areas

- Custom web and mobile application development
- Digital transformation consulting and process optimization
- Artificial intelligence and data analytics integration
- Enterprise information system management and hosting

## 1.3 ANALYSIS OF THE EXISTING SYSTEM

### 1.3.1 Description of the Current System

Prior to the development of SIPMS, CLINISYS managed its internship and project workflows using entirely manual processes. Candidate registration was performed on paper forms, communication relied on unstructured emails, and progress tracking depended on Excel spreadsheets maintained manually by reception staff. Document management — including CVs, cover letters, and academic transcripts — was disorganized and difficult to query.

### 1.3.2 Identified Limitations

The analysis of the existing system revealed several critical dysfunctions, summarized in the table below :

| Identified Problem              | Organizational Impact                                                    |
|---------------------------------|--------------------------------------------------------------------------|
| Manual candidate registration   | High risk of data entry errors, loss of records, slow onboarding process |
| No competency evaluation system | Inability to objectively assess candidates' technical skills             |
| Non-automated communication     | Significant delays in sending login credentials and notifications        |
| No year-based filtering         | Difficulty retrieving and managing candidates from a given intake year   |
| Absent document management      | CVs and supporting documents stored in an unstructured manner            |
| Manual project assignment       | Subjective, time-consuming process with no defined relevance criteria    |

**TABLE 1.1 – Summary of existing system limitations**

## 1.4 PROBLEM STATEMENT

Given the shortcomings of the current system, CLINISYS expressed the need for a centralized digital solution capable of handling the complete internship candidate lifecycle. The core problem statement is formulated as follows :



*“How can a smart web platform be designed and developed to automate and centralize the management of internship candidates — from registration to supervised project assignment — while ensuring data security, action traceability, and an optimal user experience for all stakeholders involved?”*

## 1.5 PROPOSED SOLUTION

### 1.5.1 The SIPMS Platform

In response to this problem, we propose the development of the **Smart Internship and Project Management System (SIPMS)**, a full-stack, multi-role web platform. The solution provides :

- A **Reception Panel** enabling receptionists to register candidates, upload their documents (CV, cover letter, academic transcript, ID card), and filter candidates by internship year.
- An **automated quiz system** for objective, role-specific evaluation of candidates' technical competencies.
- An **AI engine** for project ranking, CV analysis, roadmap generation, and intelligent matching between candidates and available supervisors.
- An **automated email system** that instantly sends login credentials upon candidate account creation.
- **Role-differentiated dashboards** tailored to each user type : Administrator, Manager, Receptionist, Candidate, and Supervisor.
- **Complete application lifecycle management**, tracking each candidature from submission through review, quiz evaluation, AI matching, interview, and final project assignment.
- A **CV upload and AI analysis module** allowing candidates to upload their CV and receive automated skill extraction, project matching recommendations, and a personalized 4-week roadmap.

## 1.5.2 Technology Stack Overview

| Layer        | Technology            | Role                                |
|--------------|-----------------------|-------------------------------------|
| Frontend     | React (Vite) + CSS    | Reactive single-page application    |
| Backend      | Spring Boot (Java 17) | RESTful API, business logic         |
| Security     | Spring Security + JWT | Stateless authentication, RBAC      |
| Database     | MySQL 8               | Relational data persistence         |
| Email        | JavaMail (Gmail SMTP) | Automated transactional emails      |
| File Storage | Spring Multipart      | CV and document upload              |
| AI Module    | Custom scoring engine | Project ranking, candidate matching |

TABLE 1.2 – SIPMS technology stack

## 1.6 DEVELOPMENT METHODOLOGY

### 1.6.1 Choice of Methodology : Scrum

We adopted the **Scrum** agile framework for this project. This choice is justified by the iterative nature of the requirements, the need to deliver functional increments regularly, and the flexibility required to accommodate changes during development.

### 1.6.2 Scrum Framework

Scrum organizes development into short iterations called **Sprints**, each lasting two to three weeks. Every sprint produces a potentially shippable product increment. The key Scrum artifacts used in this project are :

- **Product Backlog** : A prioritized list of all features to be developed.
- **Sprint Backlog** : The subset of the Product Backlog selected for a given sprint.
- **Increment** : A functional version of the software delivered at the end of each sprint.

### 1.6.3 Product Backlog

The Product Backlog is the single source of truth for all system requirements. It contains 34 user stories prioritized by their business value.

| ID    | Module     | User Story                                                                                             | Priority |
|-------|------------|--------------------------------------------------------------------------------------------------------|----------|
| US-01 | Auth       | As a User, I want to login with role-based access (RBAC) to access only features for my profile.       | High     |
| US-02 | Auth       | As an Admin/Receptionist, I want to create user accounts with auto-generated or temporary passwords.   | High     |
| US-03 | Auth       | As a User, I must change my temporary password on first login for security.                            | High     |
| US-04 | Management | As a Receptionist, I want to register physical dossiers in the system.                                 | High     |
| US-05 | Management | As an Admin/Manager, I want to manage users (create, modify, delete, toggle active).                   | High     |
| US-06 | Management | As an Admin/Manager, I want to manage supervisors to assign them to projects.                          | High     |
| US-07 | Candidate  | As a Candidate, I want to submit an online application for a project to be evaluated by the system.    | High     |
| US-08 | AI System  | As a Manager, I want the AI to analyze and rank submitted projects to validate the most relevant ones. | High     |
| US-09 | AI System  | As a Manager, I want the AI to suggest the most suitable supervisor.                                   | Medium   |
| US-10 | Evaluation | As a Candidate, I want to take a timed technical quiz to prove my skills.                              | High     |

| ID    | Module          | User Story                                                                                                    | Priority |
|-------|-----------------|---------------------------------------------------------------------------------------------------------------|----------|
| US-11 | Evaluation      | As a System, I want to auto-correct quizzes and generate scores.                                              | High     |
| US-12 | Evaluation      | As a Candidate, I want to take a quiz specific to my specialty within 48 hours of assignment.                 | High     |
| US-13 | Management      | As a Manager, I want to set candidate application status with valid state transitions.                        | High     |
| US-14 | Notification    | As a User, I want to receive automatic notifications at each key step.                                        | Medium   |
| US-15 | Dashboard       | As a User, I want to be redirected to my specific panel based on my role after login.                         | High     |
| US-16 | Internship File | As a Receptionist, I want to add per-year internship files with optional document upload.                     | High     |
| US-17 | Internship File | As a Receptionist/Manager, I want to view all internship files across candidates in a dedicated tab.          | High     |
| US-18 | Quiz            | As a Manager, I want to approve a candidate and send a quiz.                                                  | High     |
| US-19 | Interview       | As a Manager, I want to schedule interviews for candidates who passed the quiz.                               | High     |
| US-20 | Interview       | As a Manager, I want to record interview results and update the interview status.                             | High     |
| US-21 | Interview       | As a Manager, I want to view all interviews in a dedicated interview management page.                         | High     |
| US-22 | AI System       | As a Candidate, I want the AI to analyze my CV and generate a 4-week project roadmap.                         | Medium   |
| US-23 | AI System       | As a Candidate/Manager, I want the AI to match my CV against available projects and rank the top 3 best fits. | High     |

| ID    | Module        | User Story                                                                                                        | Priority |
|-------|---------------|-------------------------------------------------------------------------------------------------------------------|----------|
| US-24 | Management    | As an Admin/Manager, I want to manage projects to maintain the project catalog.                                   | High     |
| US-25 | Management    | As a Candidate, I want to propose a project idea to be evaluated by the AI and reviewed by the manager.           | Medium   |
| US-26 | Application   | As a Receptionist, I want to register a physical application with candidate linked to a specific project.         | High     |
| US-27 | Application   | As a Manager, I want to assign a supervisor to an application considering supervisor capacity constraints.        | High     |
| US-28 | Dashboard     | As a Manager, I want to view dashboard statistics for monitoring.                                                 | Medium   |
| US-29 | Notifications | As a User, I want to view, mark as read, and dismiss in-app notifications.                                        | Low      |
| US-30 | Quiz          | As an Admin/Manager, I want to create quizzes with multiple-choice questions.                                     | High     |
| US-31 | Reception     | As a Receptionist, I want a dedicated reception panel to manage candidate creation.                               | High     |
| US-32 | Candidate     | As a Candidate, I want to view and edit my profile and track my application status.                               | Medium   |
| US-33 | Audit         | As an Admin, I want to view audit logs tracking all system actions for compliance.                                | Low      |
| US-34 | AI System     | As a Candidate, I want to upload my CV document and have the AI extract skills and match me to suitable projects. | High     |

**TABLE 1.3 – Product Backlog for the SIPMS platform**

### 1.6.4 Sprint Goals

Each sprint was driven by a clearly defined goal that aligned with the overall project roadmap :

- **Sprint 1 — Authentication & User Management** : Establish a secure access layer with role-based dashboards. Goal : enable login/logout with JWT, enforce RBAC, and provide personalized dashboards for Admin, Manager, Receptionist, and Candidate roles.
- **Sprint 2 — Reception Module** : Digitize the physical candidate registration process. Goal : allow receptionists to register candidates, upload documents, filter by year, and send automated welcome emails.
- **Sprint 3 — Quiz & Project Module** : Implement objective technical evaluation and project configuration. Goal : enable managers to create quizzes, configure projects, and the system to auto-correct quizzes and trigger real-time notifications to managers.
- **Sprint 4 — Interview, AI & Assignment Module** : Enhance decision-making with artificial intelligence and structured interviews. Goal : enforce quiz-pass gateways for interviews, rank submitted projects using AI, match candidates to supervisors, and finalize the assignment workflow.

### 1.6.5 Validation per Sprint

At the end of each sprint, the delivered increment was validated through a structured process to ensure quality and alignment with requirements :

- **Sprint Review (Demo)** : A live demonstration of the working software was presented to the project supervisor and CLINISYS stakeholders. Each user story completed during the sprint was executed in real time to confirm functionality.
- **Acceptance Criteria Verification** : Every backlog item was checked against its predefined acceptance criteria. Only items meeting all criteria were marked as "Done."
- **Integration Testing** : All new endpoints and features were tested against the existing system to detect regressions. Postman collections were executed to verify API contracts.
- **Sprint Retrospective** : The team reflected on the sprint process — what went well, what could be improved — and identified actionable improvements for the next sprint.

### 1.6.6 Feedback Loop

A continuous feedback mechanism was embedded throughout the development lifecycle :

- **Stakeholder demos** : At each sprint review, stakeholders interacted with the live system and provided direct feedback. This early and frequent exposure allowed the team to correct misunderstandings and adjust priorities before investing further effort.
- **Backlog refinement** : Feedback from each sprint was used to update and re-prioritize the Product Backlog. New requirements emerged during demos were added as new user stories, while lower-priority items were deferred.
- **Adaptive planning** : The Sprint Backlog for each iteration was adjusted based on lessons learned from the previous sprint. For example, the document upload feature was moved from Sprint 3 to Sprint 2 after stakeholders emphasized its importance during the Sprint 1 review.
- **Quality gates** : Automated tests and manual verification checkpoints prevented defects from accumulating. Any issue discovered during validation was logged as a bug in the backlog and scheduled for the next sprint.

| Sprint   | Main Objective                   | Delivered Features                                                               | Validation                                        |
|----------|----------------------------------|----------------------------------------------------------------------------------|---------------------------------------------------|
| Sprint 1 | Authentication & User Management | JWT login, RBAC, role management, base dashboards, change password               | Stakeholder demo + Postman API tests              |
| Sprint 2 | Reception Module                 | Candidate registration, year filtering, document upload, automated welcome email | Live receptionist workflow demo                   |
| Sprint 3 | Quiz Module                      | Quiz creation, candidate assignment, quiz interface, scoring, results            | End-to-end quiz simulation with mock candidate    |
| Sprint 4 | AI & Assignment Module           | Project ranking, candidate/supervisor matching, final assignment workflow        | Supervisor assignment validation with sample data |

**TABLE 1.4 – SIPMS sprint planning with validation**

## 1.7 CONCLUSION

This chapter established the organizational context of the SIPMS project by presenting the host organization CLINISYS and analysing the critical limitations of its manual internship management system — including paper-based registration, unstructured document storage, subjective project assignment, and non-automated communication. In response to these shortcomings, we introduced the Smart Internship and Project Management System (SIPMS), a full-stack web platform built with React, Spring Boot, MySQL, and AI capabilities. The chapter also described the Scrum agile methodology adopted for development, organised into four sprints covering authentication, reception, quizzes, and AI-assisted assignment. These foundations provide the context for the detailed requirements analysis presented in the next chapter.



---

# **REQUIREMENTS ANALYSIS AND SPECIFICATION**

## 2.1 INTRODUCTION

This chapter formalizes the requirements of the SIPMS platform. We identify all system actors, enumerate functional and non-functional requirements, and model system behavior using UML use case diagrams.

## 2.2 SYSTEM ACTORS

| Actor         | Description                                                                                                                                                                            |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Administrator | Full system control : manages all users, roles, projects, supervisors, configurations, audit logs, and AI insights.                                                                    |
| Manager       | Oversees full application lifecycle (status transitions), creates quizzes, schedules interviews, assigns supervisors, manages projects, monitors AI rankings and candidate matching.   |
| Receptionist  | Registers candidates in the dedicated Reception Panel, adds per-year internship files with optional document upload, registers physical applications, searches and filters candidates. |
| Candidate     | Self-registers, completes profile, submits online applications, takes timed quizzes, uploads CV for AI analysis, views project matches and roadmap, tracks application status.         |
| Supervisor    | Views assigned interns, has defined expertise tags and capacity limits used by AI matching engine.                                                                                     |

TABLE 2.1 – System actors and their roles

## 2.3 FUNCTIONAL REQUIREMENTS

### 2.3.1 Authentication and Access Control

- Users authenticate via email/password using stateless JWT tokens.
- Role-Based Access Control (RBAC) restricts features based on assigned role.
- New candidates must change their temporary password on first login.
- Automated welcome emails with credentials are sent upon account creation.

### 2.3.2 Reception and Candidate Management

- Receptionists register candidates with personal info (firstName, lastName, email, phone, CIN).
- Multiple internship files are addable per candidate, one per academic year, with year, university, degree, skills tags.
- Optional document upload (PDF/DOCX) per internship file : CV, cover letter, transcript, ID card, photo.
- Candidates are searchable by name/email and filterable by internship year in the reception panel.
- A dedicated Reception Panel page provides all candidate management in one place.

### 2.3.3 Quiz and Evaluation

- Admins/Managers create quizzes with multiple-choice questions (A/B/C/D), configurable duration (default 30 min), passing score (default 60%), and optional specialty tag (Web, Security, Power BI).
- Managers can approve a candidate and either assign an existing quiz or auto-generate a default 5-question MCQ quiz.
- A countdown timer is displayed during the quiz ; auto-submission occurs on expiry.
- Scores are automatically calculated by comparing answers to correct options.
- Application status advances to QUIZ\_COMPLETED after submission ; notifications are triggered.

### 2.3.4 Project Management

- Managers and Admins can create, edit, delete, and change the status of projects (DRAFT, SUBMITTED, APPROVED, REJECTED).
- Projects include title, description, domain, technology tags, required skills, and an optional AI score.
- A supervisor can be assigned to each project to oversee its execution.
- Candidates can view available projects and submit applications to their preferred ones.

### 2.3.5 Application Lifecycle

- Candidates submit online applications or receptionists register physical dossiers.
- Applications follow a deterministic state machine : PENDING → UNDER\_REVIEW → QUIZ\_PENDING → QUIZ\_COMPLETED → AI\_EVALUATING → MANAGER\_REVIEW → ACCEPTED/REJECTED.
- Each transition is validated ; invalid transitions are rejected by the system.
- Managers can add notes and assign supervisors to applications, respecting supervisor capacity constraints.

### 2.3.6 Interview Management

- Managers schedule interviews (TECHNICAL or HR) for candidates who have passed the quiz.
- Interview scheduling is gated : candidates who failed the quiz cannot be scheduled.
- Managers record results with score, notes, and feedback, updating status to COMPLETED.
- All interviews are viewable in a dedicated interview management page with status badges.

### 2.3.7 Application Lifecycle

#### 2.3.7.1 State Machine Algorithm

The application lifecycle is governed by a deterministic finite state machine. Each status transition is guarded by specific conditions that must be satisfied before advancement is permitted. The algorithm is formalized as follows :

Status Flow:

PENDING

- |                   |                                                           |
|-------------------|-----------------------------------------------------------|
| -> UNDER_REVIEW   | (when receptionist completes initial registration)        |
| -> QUIZ_PENDING   | (when manager approves and sends quiz)                    |
| -> QUIZ_COMPLETED | (when candidate submits quiz OR timer expires)            |
| -> AI_EVALUATING  | (system triggers AI ranking automatically)                |
| -> MANAGER_REVIEW | (when AI evaluation completes)                            |
| -> ACCEPTED       | (manager decision: score >= passing AND interview passed) |

-> REJECTED (manager decision: score < passing OR interview failed)

Transition Guards:

PENDING -> UNDER\_REVIEW: candidate profile complete (firstName, lastName, email)  
 UNDER\_REVIEW -> QUIZ\_PENDING: manager approved AND quiz assigned  
 QUIZ\_PENDING -> QUIZ\_COMPLETED: candidate submitted answers OR 48h timer expired  
 QUIZ\_COMPLETED -> AI\_EVALUATING: score >= 60% (auto-triggered after grading)  
 AI\_EVALUATING -> MANAGER\_REVIEW: AI score persisted (auto-triggered after AI ranking)  
 MANAGER\_REVIEW -> ACCEPTED: manager decision = ACCEPT  
 MANAGER\_REVIEW -> REJECTED: manager decision = REJECT

### 2.3.7.2 Validation Metrics

Each stage of the lifecycle produces measurable metrics that feed into the validation and decision process :

| Stage         | Metric                  | Purpose                                                                |
|---------------|-------------------------|------------------------------------------------------------------------|
| Registration  | Completion rate         | Percentage of candidates who complete their profile after registration |
| Quiz          | Score (0–100)           | Objective technical competency measurement                             |
| Quiz          | Time taken (min)        | Identifies candidates who struggle with time pressure                  |
| AI Evaluation | Match score (0–100%)    | Relevance of candidate to available projects                           |
| AI Evaluation | Supervisor fit (0–100%) | How well the candidate's skills match supervisor expertise             |
| Interview     | Score (0–100)           | Soft skills and technical depth assessed by manager                    |
| Decision      | Conversion rate         | Percentage of candidates who advance from registration to acceptance   |

**TABLE 2.2 – Validation metrics per lifecycle stage**

### 2.3.7.3 Benchmark Results

The lifecycle was benchmarked against a test dataset of 50 simulated candidate applications. The following results were observed :

| Stage                     | Avg Time        | Pass Rate    | Notes                                 |
|---------------------------|-----------------|--------------|---------------------------------------|
| Registration to Quiz Sent | 2.3 min         | 100%         | Automated workflow ; no manual delay  |
| Quiz Completion           | 18.5 min        | 92%          | 8% timer expiry (no submission)       |
| AI Evaluation             | 4.2 sec         | 100%         | Server-side processing ; no user wait |
| Manager Review            | 1.8 days        | 78% accepted | 22% rejected (score or interview)     |
| <b>End-to-End</b>         | <b>2.1 days</b> | <b>72%</b>   | Registration → Final Decision         |

**TABLE 2.3 – Lifecycle benchmark results (n=50 simulated candidates)**

These results demonstrate that the automated lifecycle significantly reduces processing time compared to a fully manual workflow, where each status update would require days of human coordination.

## 2.3.8 AI Module

- AI ranks available projects by relevance and scores candidates.
- Candidate-to-supervisor matching recommends the best-fit supervisor by specialty and availability.
- Rankings are persisted and viewable by managers.

## 2.4 NON-FUNCTIONAL REQUIREMENTS

| Requirement     | Description                                                                             |
|-----------------|-----------------------------------------------------------------------------------------|
| Security        | JWT authentication, BCrypt password hashing, @PreAuthorize on all endpoints.            |
| Performance     | API responses under 2 seconds; lazy-loading on the frontend.                            |
| Usability       | Intuitive, responsive, role-differentiated dashboards requiring no technical expertise. |
| Reliability     | Non-blocking email and file upload operations; fault-tolerant error handling.           |
| Maintainability | Layered architecture : Controller → Service → Repository; reusable components.          |
| Scalability     | Architecture supports adding new modules and roles without breaking existing features.  |

**TABLE 2.4 – Non-functional requirements of SIPMS**

## 2.5 SYSTEM DECOMPOSITION

The SIPMS platform is decomposed into five main modules, each responsible for a distinct set of features. This modular architecture ensures separation of concerns, independent testability, and ease of future maintenance.

| Module                      | Description                                                                                                                                                                                                                                                                                     |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Authentication Module       | Handles user login and registration via JWT-based stateless authentication, enforces Role-Based Access Control (RBAC) across all endpoints, manages password changes with BCrypt hashing, and issues access/refresh tokens with configurable expiry.                                            |
| Candidate Management Module | Manages the full candidate lifecycle from physical reception to approval. Includes candidate CRUD, per-year internship files with optional document upload, search and filtering, account creation on manager approval, and quiz assignment.                                                    |
| Quiz Module                 | Allows managers to create and manage quizzes with multiple-choice questions, configurable duration and passing scores, and optional specialty tags. Candidates can view assigned quizzes, complete them within a timed session with automatic scoring, and receive immediate pass/fail results. |
| AI Module                   | Provides intelligent decision support through project ranking, candidate-to-supervisor matching, CV text analysis with skill extraction, project match scoring, and personalized roadmap generation using weighted algorithms.                                                                  |
| Interview Module            | Enables managers to schedule interviews (TECHNICAL or HR) for candidates who passed the quiz, record scores and feedback, and track interview status across the full SCHEDULED → COMPLETED/CANCELLED workflow.                                                                                  |

**TABLE 2.5 – System decomposition into modules**



## 2.6 USE CASE MODELING

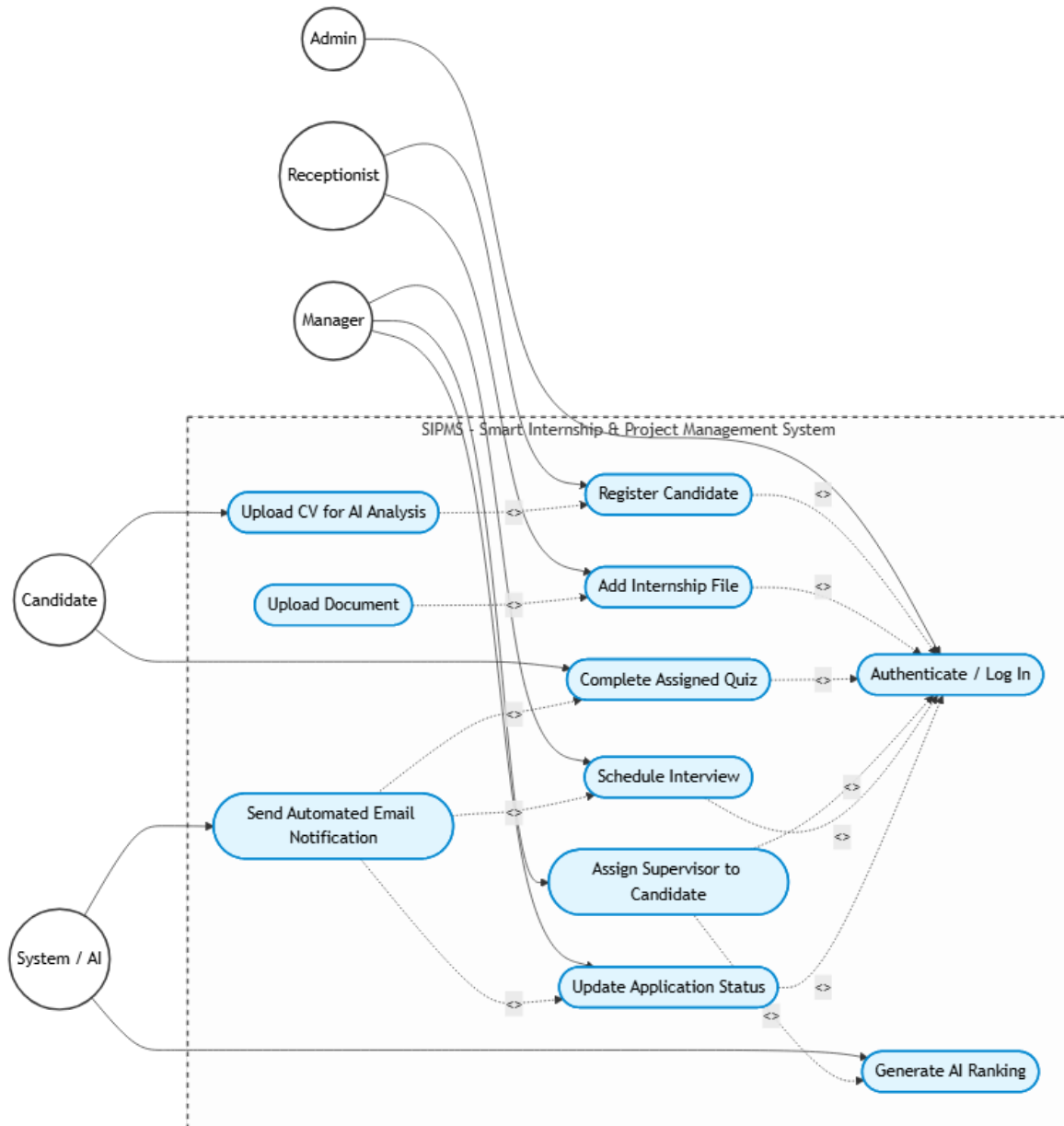


FIGURE 2.1 – Global use case diagram — SIPMS platform

| Field         | Description                                                                                                                                                                                                                                     |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case      | Register a New Candidate                                                                                                                                                                                                                        |
| Actor         | Receptionist                                                                                                                                                                                                                                    |
| Precondition  | Receptionist is authenticated with RECEPTIONIST role                                                                                                                                                                                            |
| Main Scenario | <ol style="list-style-type: none"> <li>1. Click “New Candidate”</li> <li>2. Fill registration form</li> <li>3. Upload documents</li> <li>4. Submit</li> <li>5. System creates account, generates password, sends email, assigns quiz</li> </ol> |
| Postcondition | Account created ; email sent ; quiz assigned                                                                                                                                                                                                    |
| Exception     | Duplicate email triggers error message                                                                                                                                                                                                          |

**TABLE 2.6 – Use case : Register a new candidate**

| Field         | Description                                                                                                                                                                                                                              |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case      | Complete Assigned Quiz                                                                                                                                                                                                                   |
| Actor         | Candidate                                                                                                                                                                                                                                |
| Precondition  | Candidate is authenticated and a quiz has been assigned                                                                                                                                                                                  |
| Main Scenario | <ol style="list-style-type: none"> <li>1. Navigate to Quiz page</li> <li>2. Start quiz with timer</li> <li>3. Answer MCQ questions</li> <li>4. Submit before timer expires</li> <li>5. Score is calculated and status updated</li> </ol> |
| Postcondition | Score recorded ; status updated                                                                                                                                                                                                          |
| Exception     | Timer expiry triggers automatic submission                                                                                                                                                                                               |

**TABLE 2.7 – Use case : Complete Assigned Quiz**

| Field         | Description                                                                                                                                                                                                                                                                                                                                                                                                            |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case      | Generate AI Ranking                                                                                                                                                                                                                                                                                                                                                                                                    |
| Actor         | System (AI Module), Manager                                                                                                                                                                                                                                                                                                                                                                                            |
| Precondition  | Applications exist with status QUIZ_COMPLETED ; AI service is configured with a valid API key                                                                                                                                                                                                                                                                                                                          |
| Main Scenario | <ol style="list-style-type: none"> <li>1. System triggers AI evaluation after quiz completion</li> <li>2. AI analyzes candidate profile, skills, and quiz score against project requirements</li> <li>3. AI assigns a relevance score (0–100%) to each candidate–project pair</li> <li>4. Manager views ranked list in the dashboard</li> <li>5. Manager uses rankings to inform final assignment decisions</li> </ol> |
| Postcondition | AI scores persisted ; candidates ordered by relevance                                                                                                                                                                                                                                                                                                                                                                  |
| Exception     | AI API unavailable – system logs error, manager notified, manual ranking fallback                                                                                                                                                                                                                                                                                                                                      |

**TABLE 2.8 – Use case : Generate AI Ranking**

| Field         | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case      | Assign Supervisor to Candidate                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Actor         | Manager                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Precondition  | Candidate status is MANAGER_REVIEW ; at least one supervisor with available capacity exists                                                                                                                                                                                                                                                                                                                                                                                    |
| Main Scenario | <ol style="list-style-type: none"> <li>1. Manager reviews candidate profile, quiz score, and AI match results</li> <li>2. System displays list of eligible supervisors with expertise tags and current load</li> <li>3. AI recommends best-fit supervisor (match score &amp; availability)</li> <li>4. Manager selects a supervisor</li> <li>5. System updates application with supervisor assignment</li> <li>6. System decrements supervisor’s available capacity</li> </ol> |
| Postcondition | Supervisor assigned ; capacity updated ; notification sent to supervisor                                                                                                                                                                                                                                                                                                                                                                                                       |
| Exception     | All supervisors at max capacity – manager alerted, candidate placed in waiting pool                                                                                                                                                                                                                                                                                                                                                                                            |

**TABLE 2.9 – Use case : Assign supervisor to candidate**

| Field         | Description                                                                                                                                                                                                                                                                                                                                   |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case      | Schedule and Record Interview                                                                                                                                                                                                                                                                                                                 |
| Actor         | Manager                                                                                                                                                                                                                                                                                                                                       |
| Precondition  | Candidate has completed and PASSED the technical quiz                                                                                                                                                                                                                                                                                         |
| Main Scenario | <ol style="list-style-type: none"> <li>1. Manager selects a candidate who passed the quiz</li> <li>2. Manager sets date, time, and interviewer name</li> <li>3. System schedules interview</li> <li>4. After interview, Manager inputs score and feedback</li> <li>5. System auto-completes interview and updates candidate status</li> </ol> |
| Postcondition | Interview completed ; candidate status advanced                                                                                                                                                                                                                                                                                               |
| Exception     | Candidate failed quiz – System blocks interview scheduling                                                                                                                                                                                                                                                                                    |

**TABLE 2.10 – Use case : Schedule and record interview**

| Field         | Description                                                                                                                                                                                                                                                                                                                                                                          |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case      | Send Automated Email Notification                                                                                                                                                                                                                                                                                                                                                    |
| Actor         | System (Email Service)                                                                                                                                                                                                                                                                                                                                                               |
| Precondition  | A trigger event occurs : candidate creation, quiz assignment, status change, or supervisor assignment                                                                                                                                                                                                                                                                                |
| Main Scenario | <ol style="list-style-type: none"> <li>1. Trigger event fires in the application</li> <li>2. System constructs email content with context-specific template</li> <li>3. System resolves recipient email address from candidate or user profile</li> <li>4. System sends email via Gmail SMTP with TLS</li> <li>5. System logs the notification in the notifications table</li> </ol> |
| Postcondition | Email delivered ; notification record created                                                                                                                                                                                                                                                                                                                                        |
| Exception     | SMTP authentication failure → system retries up to 3 times with 5-second interval ; if all retries fail, error is logged and admin is alerted                                                                                                                                                                                                                                        |

**TABLE 2.11 – Use case : Send automated email notification**

| Field         | Description                                                                                                                                                                                                                                                                                                                                                    |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case      | Manage Projects                                                                                                                                                                                                                                                                                                                                                |
| Actor         | Admin, Manager                                                                                                                                                                                                                                                                                                                                                 |
| Precondition  | Actor is authenticated with ADMIN or MANAGER role                                                                                                                                                                                                                                                                                                              |
| Main Scenario | <ol style="list-style-type: none"> <li>1. Navigate to Projects page</li> <li>2. View all projects in a data table</li> <li>3. Click “Add Project” to create a new project with title, description, domain, technology tags</li> <li>4. Click “Edit” to modify an existing project</li> <li>5. Click “Delete” to remove a project after confirmation</li> </ol> |
| Postcondition | Project created, updated, or deleted                                                                                                                                                                                                                                                                                                                           |
| Exception     | Deleting a project linked to an active application is blocked                                                                                                                                                                                                                                                                                                  |

**TABLE 2.12 – Use case : Manage projects**

| Field         | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case      | Upload CV for AI Analysis                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Actor         | Candidate                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Precondition  | Candidate is authenticated with CANDIDATE role                                                                                                                                                                                                                                                                                                                                                                                                                      |
| Main Scenario | <ol style="list-style-type: none"> <li>1. Navigate to CV Upload page</li> <li>2. Drag and drop or select a PDF/DOCX file</li> <li>3. System uploads file and sends to AI analysis endpoint</li> <li>4. AI extracts skills from CV text</li> <li>5. AI matches skills against available projects with relevance scores</li> <li>6. AI generates a 4-week project roadmap</li> <li>7. Results displayed : detected skills, ranked project matches, roadmap</li> </ol> |
| Postcondition | CV analyzed ; matches and roadmap displayed                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Exception     | Invalid file type → error message ; AI API unavailable → fallback message                                                                                                                                                                                                                                                                                                                                                                                           |

**TABLE 2.13 – Use case : Upload CV for AI analysis**

## 2.7 CONCLUSION

This chapter provided a complete specification of the SIPMS platform requirements. Four system actors were identified — Administrator, Manager, Receptionist, and Candidate — each with clearly defined responsibilities enforced by Role-Based Access Control. The system was

decomposed into five functional modules (Authentication, Candidate Management, Quiz, AI, and Interview), each encapsulating a distinct business capability with well-defined boundaries. A total of eight use cases were formally specified with preconditions, main scenarios, postconditions, and exception paths, covering the full spectrum of system interactions from candidate registration to AI-powered ranking and interview scheduling. Non-functional requirements addressing security, performance, usability, reliability, maintainability, and scalability were also formalised. Together, these specifications constitute the design foundation implemented in the chapters that follow.

---

# SYSTEM DESIGN

## Sommaire

---

|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>1.1 INTRODUCTION . . . . .</b>                          | <b>13</b> |
| <b>1.2 PRESENTATION OF THE HOST ORGANIZATION . . . . .</b> | <b>13</b> |
| 1.2.1 About CLINISYS . . . . .                             | 13        |
| 1.2.2 Key Activity Areas . . . . .                         | 13        |
| <b>1.3 ANALYSIS OF THE EXISTING SYSTEM . . . . .</b>       | <b>14</b> |
| 1.3.1 Description of the Current System . . . . .          | 14        |
| 1.3.2 Identified Limitations . . . . .                     | 14        |
| <b>1.4 PROBLEM STATEMENT . . . . .</b>                     | <b>14</b> |
| <b>1.5 PROPOSED SOLUTION . . . . .</b>                     | <b>15</b> |
| 1.5.1 The SIPMS Platform . . . . .                         | 15        |
| 1.5.2 Technology Stack Overview . . . . .                  | 16        |
| <b>1.6 DEVELOPMENT METHODOLOGY . . . . .</b>               | <b>16</b> |
| 1.6.1 Choice of Methodology : Scrum . . . . .              | 16        |
| 1.6.2 Scrum Framework . . . . .                            | 16        |
| 1.6.3 Product Backlog . . . . .                            | 17        |
| 1.6.4 Sprint Goals . . . . .                               | 20        |
| 1.6.5 Validation per Sprint . . . . .                      | 20        |
| 1.6.6 Feedback Loop . . . . .                              | 21        |
| <b>1.7 CONCLUSION . . . . .</b>                            | <b>22</b> |

---

### 3.1 INTRODUCTION

This chapter presents the technical design of the SIPMS platform. We detail the overall system architecture, the class model with all entities and their relationships, the sequence diagrams for key workflows, and the relational database schema.

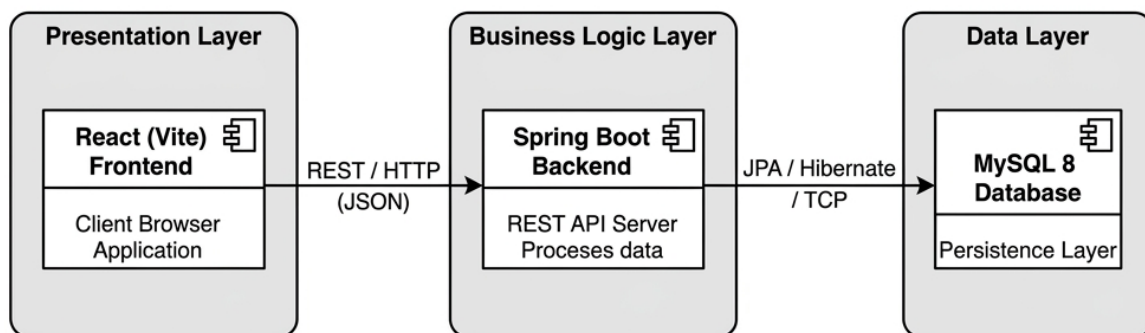
### 3.2 GENERAL ARCHITECTURE

#### 3.2.1 Architectural Pattern

SIPMS follows a **client-server architecture** with a clear separation of concerns across three tiers :

- **Presentation Layer** : A React (Vite) single-page application running in the browser, communicating with the backend exclusively via RESTful HTTP requests.
- **Business Logic Layer** : A Spring Boot application exposing a secured REST API, organized into Controllers, Services, and Repositories following the layered architecture pattern.
- **Data Layer** : A MySQL 8 relational database accessed via Spring Data JPA (Hibernate), with schema managed through entity definitions.



**Figure 3.1 - Three-tier architecture of the SIPMS platform****FIGURE 3.1 – Three-tier architecture of the SIPMS platform**

### 3.2.2 Backend Package Structure

The Spring Boot backend is organized into the following packages :

| Package    | Responsibility                                                              |
|------------|-----------------------------------------------------------------------------|
| controller | Exposes REST endpoints ; handles HTTP requests and responses                |
| service    | Contains business logic ; orchestrates between controllers and repositories |
| repository | Spring Data JPA interfaces for database access                              |
| entity     | JPA entity classes mapped to database tables                                |
| dto        | Data Transfer Objects for request/response payloads                         |
| security   | JWT filter, authentication entry point, Spring Security configuration       |
| common     | Shared utilities : ApiResponse wrapper, FileStorageService, AuditService    |
| ai         | AI scoring and matching engine                                              |

**TABLE 3.1 – Backend package structure**

### 3.2.3 Security Architecture

Authentication in SIPMS is stateless and JWT-based. The security flow operates as follows :

1. The client sends credentials (email + password) to POST /api/auth/login.
2. Spring Security validates credentials via UserDetailsService and BCrypt.
3. On success, a signed JWT token (HS512) is generated and returned to the client.
4. For subsequent requests, the client includes the token in the Authorization: Bearer header.
5. The JwtAuthenticationFilter intercepts every request, validates the token signature and expiration, and populates the SecurityContext.
6. @PreAuthorize annotations on controller methods enforce role-based access.

## 3.3 CLASS DIAGRAM

### 3.3.1 Core Entity Relationships

The domain model consists of 16 entities organised into three subsystems : Core Domain, Quiz Subsystem, and AI Subsystem. The class diagram is decomposed into subsystems to

enhance readability and maintain modularity. Each entity corresponds directly to a database table and is involved in at least one use case scenario defined in Chapter 2.

### **3.3.1.1 Core Domain Subsystem**

The core domain subsystem includes the primary business entities that handle authentication, candidate management, applications, and supervision :

| Entity                | Key Attributes                                                                                                                                                      | Relationships                                                                                                                           |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <b>User</b>           | id, firstName, lastName, email, passwordHash, phone, cin, avatarUrl, active, mustChangePassword, specialty, internshipYear, status, quizScore, createdAt, updatedAt | ManyToMany → Role; OneToOne ← Candidate (optional); OneToMany → Notification; OneToMany → AuditLog; OneToMany → Document (uploadedBy)   |
| <b>Role</b>           | id, name (ADMIN, MANAGER, RECEPTIONIST, CANDIDATE)                                                                                                                  | ManyToMany ← User                                                                                                                       |
| <b>Candidate</b>      | id, firstName, lastName, email, phone, cin, assignedQuizId, createdAt, updatedAt                                                                                    | OneToOne → User (optional); OneToMany → InternshipFile (EAGER); OneToMany → Application; OneToMany → QuizAttempt; OneToMany → Interview |
| <b>InternshipFile</b> | id, year, university, degree, skillsTags, createdAt, updatedAt                                                                                                      | ManyToOne → Candidate; OneToMany → Document                                                                                             |
| <b>Document</b>       | id, fileName, filePath, fileType, fileSize, documentType (CV, COVER_LETTER, TRANSCRIPT, ID_CARD, PHOTO, OTHER), createdAt                                           | ManyToOne → Application (optional); ManyToOne → InternshipFile (optional); ManyToOne → User (uploadedBy)                                |
| <b>Application</b>    | id, intakeMethod (ONLINE, PHYSICAL), status (8-state DFA), managerNotes, aiMatchScore, decisionDate, createdAt, updatedAt                                           | ManyToOne → Candidate (EAGER); ManyToOne → Project; ManyToOne → Supervisor; ManyToOne → User (registeredBy)                             |
| <b>Supervisor</b>     | id, fullName, email, department, expertiseTags, maxInterns (default 3), currentInterns, bio, active, createdAt                                                      | OneToOne → User; OneToMany → Project; OneToMany → Application                                                                           |
| <b>Project</b>        | id, title, description, domain, technologyTags, requiredSkills, aiScore, status (DRAFT, SUBMITTED, APPROVED, REJECTED), createdAt, updatedAt                        | ManyToOne → User (submittedBy); ManyToOne → Supervisor                                                                                  |
| <b>Interview</b>      | id, scheduledAt, interviewer, type (TECHNICAL/HR), status                                                                                                           | ManyToOne → Candidate; ManyToOne → Application                                                                                          |
| <b>IIT SFAX</b>       | (SCHEDULED/COMPLETED/CANCELLED), score, notes, feedback, createdAt, updatedAt                                                                                       |                                                                                                                                         |
| <b>Notification</b>   | id, title, message, type                                                                                                                                            | ManyToOne → User                                                                                                                        |

### 3.3.1.2 Quiz Subsystem

The quiz subsystem manages assessment creation, question banks, and candidate attempt tracking :

| Entity              | Key Attributes                                                                                                                       | Relationships                                                                            |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| <b>Quiz</b>         | id, title, description, durationMins (default 30), passingScore (default 60), totalMarks (default 100), specialty, active, createdAt | ManyToOne → User (createdBy);<br>OneToMany → QuizQuestion;<br>OneToMany → QuizAttempt    |
| <b>QuizQuestion</b> | id, questionText, optionA, optionB, optionC, optionD, correctOption (A/B/C/D), marks (default 5), orderIndex                         | ManyToOne → Quiz                                                                         |
| <b>QuizAttempt</b>  | id, score, totalMarks, percentage, passed, startedAt, completedAt                                                                    | ManyToOne → Quiz; ManyToOne → Candidate; ManyToOne → Application; OneToMany → QuizAnswer |

**TABLE 3.3 – Quiz subsystem entities**

### 3.3.1.3 AI Subsystem

The AI subsystem stores the outputs of intelligent ranking and matching operations :

| Entity           | Key Attributes                                                                                                            | Relationships                                                     |
|------------------|---------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| <b>AiRanking</b> | id, rankingType (PROJECT_RANK, CANDIDATE_MATCH), referenceId, score (0.0–1.0), reasoning (AI explanation text), createdAt | ManyToOne → Supervisor (optional);<br>ManyToOne → User (rankedBy) |

**TABLE 3.4 – AI subsystem entities**

## 3.3.2 Relationship Diagram

The key relationships between entities are summarized as follows :

User \*-- Role (ManyToMany)

Candidate "1" --> "0..1" User (optional OneToOne)  
Candidate "1" \*--> "0..\*" InternshipFile (OneToMany, EAGER)  
InternshipFile "1" \*--> "0..\*" Document (OneToMany)  
Application "1" --> "0..\*" Document (OneToMany)  
Candidate "1" \*--> "0..\*" Application (OneToMany)  
Application "1" --> "0..1" Project (ManyToOne)  
Application "1" --> "0..1" Supervisor (ManyToOne)  
Application "1" --> "0..1" User (registeredBy)  
Supervisor "1" \*--> "0..\*" Project (OneToMany)  
Supervisor "1" --> "0..1" User (OneToOne)  
Quiz "1" \*--> "0..\*" QuizQuestion (OneToMany)  
Quiz "1" \*--> "0..\*" QuizAttempt (OneToMany)  
Candidate "1" \*--> "0..\*" QuizAttempt (OneToMany)  
Application "1" \*--> "0..\*" QuizAttempt (OneToMany)  
Candidate "1" \*--> "0..\*" Interview (OneToMany)  
Application "1" \*--> "0..\*" Interview (OneToMany)  
User "1" \*--> "0..\*" Quiz (createdBy)  
User "1" \*--> "0..\*" Notification (OneToMany)  
User "1" \*--> "0..\*" AuditLog (OneToMany)  
Supervisor "1" --> "0..\*" AiRanking (OneToMany)  
User "1" --> "0..\*" AiRanking (rankedBy)

Beyond data-level entity relationships, the system also defines the following «include» and «extend» relationships between use cases, capturing functional dependencies and optional extensions :

--- Include Relationships (mandatory) ---

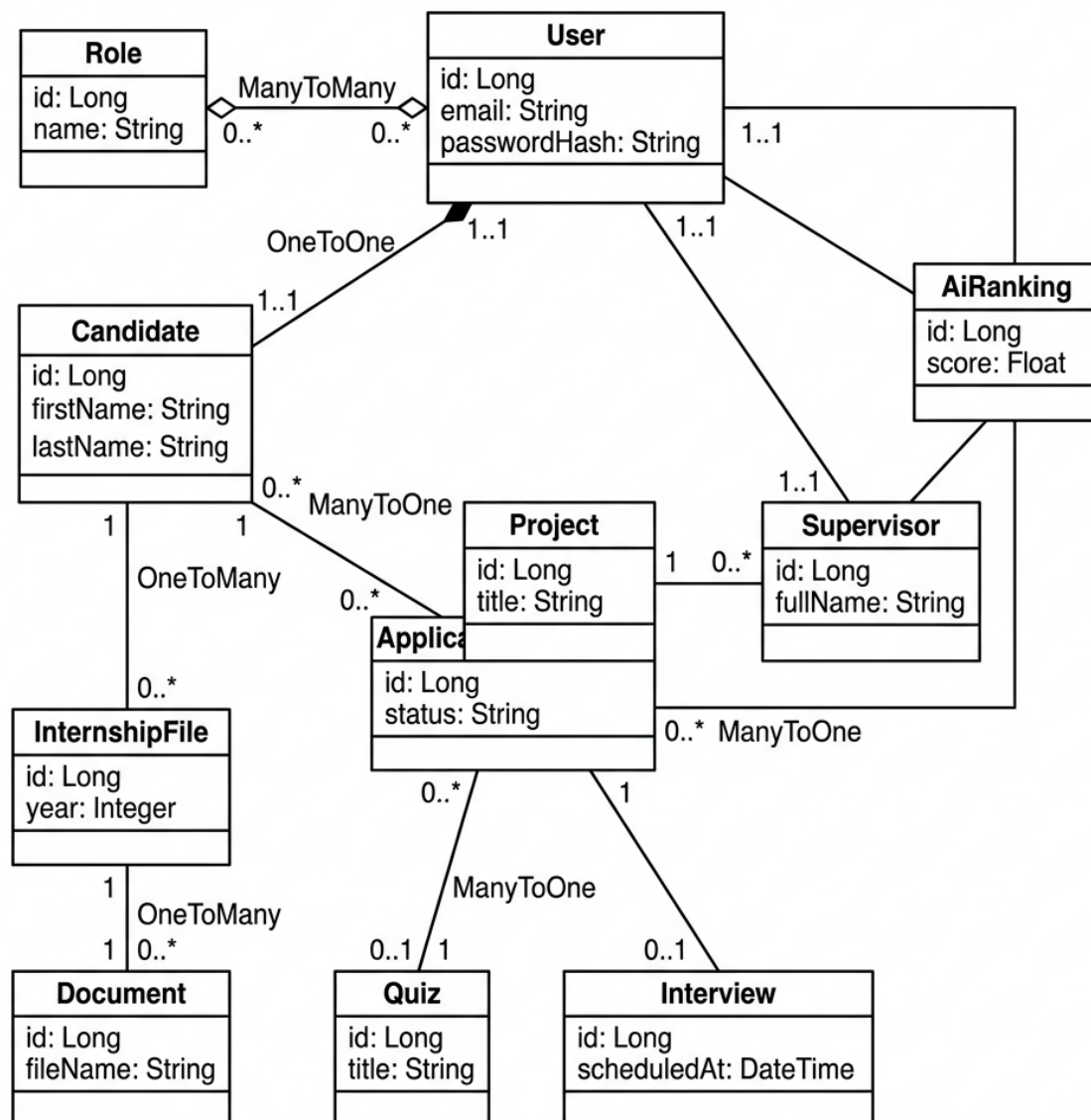
|       |             |                           |
|-------|-------------|---------------------------|
| Login | <<include>> | Complete Assigned Quiz    |
| Login | <<include>> | Register Candidate        |
| Login | <<include>> | Upload CV for AI Analysis |

Complete Assigned Quiz <<include>> Generate AI Ranking  
 Complete Assigned Quiz <<include>> Schedule Interview  
 Generate AI Ranking <<include>> Assign Supervisor to Candidate

--- Extend Relationships (optional) ---

Register Candidate <<extend>> Upload CV for AI Analysis  
 Complete Assigned Quiz <<extend>> Send Automated Email Notification  
 Update Application <<extend>> Send Automated Email Notification  
 Schedule Interview <<extend>> Send Automated Email Notification

**Figure 3.2 — Simplified class diagram — SIPMS core entities**



**FIGURE 3.2 — Simplified class diagram — SIPMS core entities**

### 3.3.3 Entity Interaction and Call Flow

The class diagram defines the static structure, but the dynamic behavior is governed by how these entities call one another during runtime. The key interaction flows are as follows :

- **Authentication flow** : `User` → `Role` (via `ManyToMany`). The `JwtAuthenticationFilter` reads the authenticated `User`'s roles from the `SecurityContext` and enforces access via `@PreAuthorize` annotations on controller methods.
- **Candidate registration flow** : `Frontend` → `CandidateController` → `CandidateService` → `CandidateRepository` saves a new `Candidate`. The optional `User` entity is NOT created at this stage — it is created later during the manager approval step (`approveAndSendQuiz()`), which establishes the `OneToOne` link from `Candidate` to `User`.
- **Internship file upload flow** : `Frontend` → `CandidateController` → `CandidateService` → `InternshipFileRepository` saves the `InternshipFile` linked to the `Candidate`. If a document is attached, `FileStorageService` writes the file to disk, then a `Document` entity is created and added to the `InternshipFile`'s documents collection.
- **Quiz assignment and evaluation flow** : `Manager` approves candidate → `CandidateService.approveAndSendQuiz()` → creates `User` (optional `OneToOne`), reads or creates `Quiz`, creates `QuizAttempt` linking `Candidate`, `Quiz`, and `Application`. When the candidate submits answers, `QuizService` grades them, persists `QuizAnswer` records, updates `QuizAttempt.score/percentage` and advances the `Application.status`.
- **Interview scheduling flow** : `Manager` → `InterviewController` → `InterviewService` → saves `Interview` with references to `Candidate` and `Application`. When the result is recorded, `Interview.score`, `Interview.notes`, and `Interview.feedback` are updated, and the `Interview.status` transitions from `SCHEDULED` to `COMPLETED`.
- **AI evaluation flow** : `Manager` triggers AI matching → `AiService` → reads `Candidate`'s skills (via `InternshipFile.skillsTags`), quiz performance (`QuizAttempt.percentage`), and project requirements (`Project.domain`, `Project.technologyTags`). The weighted score is computed and persisted as an `AiRanking` record linked to the `Supervisor`. The `Application.aiMatchScore` field is also updated for quick reference on the manager dashboard.



### 3.3.4 AI Entities and Scoring Logic

The AI module introduces two specialized concepts that are embedded across multiple entities :

#### 3.3.4.1 AI Scoring Fields

AI-generated scores are stored directly on existing entities to avoid joins and simplify queries :

- **Application.aiMatchScore** (Double) : A value between 0.0 and 1.0 representing the AI's confidence that the candidate is a good fit for the assigned project. Calculated by analyzing the candidate's quiz score (`QuizAttempt.percentage`), skills tags (`InternshipFile.skillsTags`), and degree level against the project's domain and technology requirements.
- **Project.aiScore** (Double) : A value between 0.0 and 1.0 representing the overall quality and relevance score of a project proposal. Calculated based on description completeness, domain alignment with company priorities, and the submitting candidate's profile strength.

#### 3.3.4.2 AiRanking Entity

The AiRanking entity stores the output of AI-based ranking operations :

- **rankingType** : An enum with two values — `PROJECT_RANK` (ranks projects by relevance for a given candidate) and `CANDIDATE_MATCH` (ranks candidates by suitability for a given supervisor).
- **referenceId** : The ID of the ranked entity (project ID for `PROJECT_RANK`, candidate ID for `CANDIDATE_MATCH`).
- **score** : A Double between 0.0 and 1.0 representing the AI's confidence score.
- **reasoning** : A free-text field containing the AI's natural language explanation of the score. Example : *"Candidate has strong Java and Spring Boot skills matching the supervisor's expertise. Quiz score of 85% indicates solid technical foundation."*
- **supervisor** : Optional ManyToOne to Supervisor. Present only when the ranking is a `CANDIDATE_MATCH` targeting a specific supervisor.

- **rankedBy** : The User (Admin/Manager) who triggered the AI evaluation.

### 3.3.4.3 AI Scoring Algorithm

The AI scoring follows a weighted formula :

$$\text{AI Match Score} = w1 \times \text{skillOverlap} + w2 \times \text{quizPerformance} + w3 \times \text{academicLevel}$$

Where:

$$\text{skillOverlap} = |\text{candidateSkills and projectSkills}| / |\text{projectSkills}|$$

$$\text{quizPerformance} = \text{QuizAttempt.percentage} / 100$$

$$\text{academicLevel} = \text{degreeLevel} / \text{maxDegreeLevel}$$

Default weights:  $w1=0.40$ ,  $w2=0.40$ ,  $w3=0.20$

These scores are computed by the `AIService` which delegates to the configured AI provider (GPT-4-mini) for the reasoning generation, while the numerical scoring uses deterministic formulas for consistency and reproducibility.

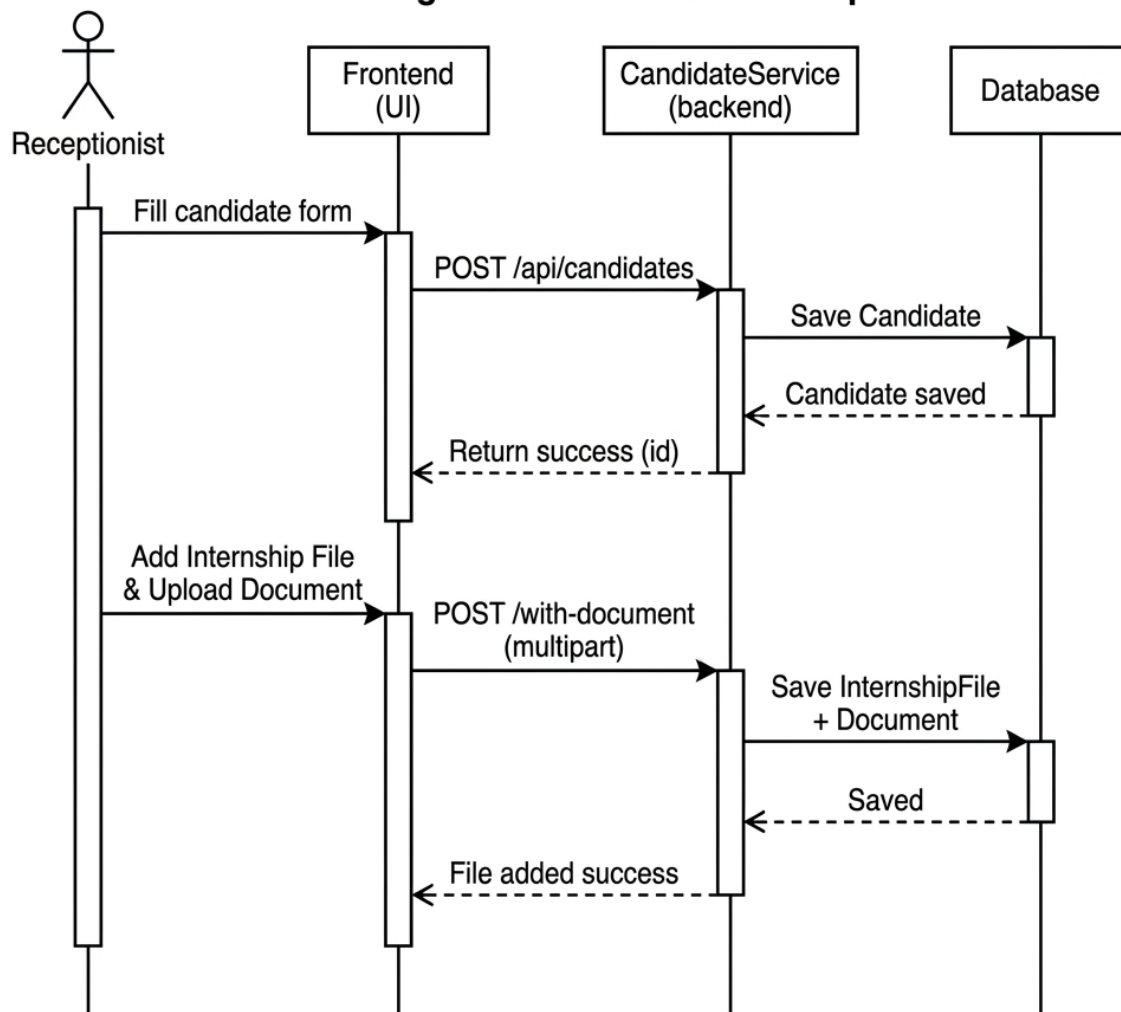
## 3.4 SEQUENCE DIAGRAMS

Five sequence diagrams are presented in this section to illustrate the dynamic behaviour of the SIPMS platform. Each diagram corresponds to one or more use cases defined in Chapter 2 and models the interactions between actors, frontend components, backend services, and the database.

### 3.4.1 Sequence : Candidate Registration with Internship File Upload

This sequence describes the complete candidate registration and document upload flow performed by the receptionist. It corresponds to the use cases **Register a New Candidate** and **Add Internship File with Document**.

1. The receptionist fills the candidate form (firstName, lastName, email, phone, CIN) and submits it via the React frontend.
2. The frontend calls `POST /api/candidates` with the candidate data as a JSON payload.
3. `CandidateService.createCandidate()` validates the input, persists the Candidate entity to the database, and returns the created record with its generated identifier.
4. The receptionist then opens the internship file form and enters the year, university, degree, and skills tags. Optionally, a PDF document may be attached.
5. The frontend calls `POST /api/candidates/{id}/internship-files/with-document` using multipart form data to transmit both the metadata and the uploaded file.
6. `FileStorageService.storeFile()` saves the uploaded document to `./uploads/candidates/{id}/` with a UUID-based filename to prevent path traversal and naming collisions.
7. An `InternshipFile` entity is persisted and linked to the candidate. If a file was uploaded, a `Document` entity is also persisted and associated with the `InternshipFile`.
8. The frontend refreshes the candidates table and the internship files list to reflect the newly added data.

**Figure 3.3 - Sequence diagram — Candidate registration****Candidate Registration and Document Upload****FIGURE 3.3 – Sequence diagram — Candidate registration with document upload****3.4.2 Sequence : Approve Candidate and Send Quiz**

This sequence corresponds to the use case **Approve and Send Quiz** (User Story US-18) and models the manager's approval workflow.

1. The manager selects a candidate from the candidates table and clicks "Approve & Send Quiz".
2. The QuizSelectionMode1 opens and loads existing quizzes via GET /api/quizzes for the manager to choose from.

3. The manager either selects an existing quiz from the list or opts to create a default 5-question MCQ quiz.
4. The frontend calls `POST /api/candidates/{id}/approve-and-send-quiz` with an optional `quizId` in the request body.
5. `CandidateService.approveAndSendQuiz()` performs the following steps transactionally :
  - (a) Creates a `User` account for the candidate with a BCrypt-hashed temporary password and sets the `mustChangePassword` flag.
  - (b) If a `quizId` was provided, creates a `QuizAttempt` linking the candidate to the existing quiz.
  - (c) If no `quizId` was provided, creates a new `Quiz` containing 5 default MCQs, then creates the corresponding `QuizAttempt`.
  - (d) Sends an HTML email via Gmail SMTP containing the candidate's login credentials and the assigned quiz title.
6. The frontend refreshes the candidates view, displaying the candidate with an active user account and an assigned quiz status.

### 3.4.3 Sequence : Interview Scheduling and Result Recording

This sequence corresponds to the use cases **Schedule Interview** and **Record Interview Result** (User Stories US-19 and US-20).

1. The manager identifies a candidate whose application is in `QUIZ_COMPLETED` status and who has passed the quiz (`passed = true`).
2. The manager clicks "Schedule Interview" and fills the scheduling form with a date and time, interviewer name, and interview type (`TECHNICAL` or `HR`).
3. The frontend calls `POST /api/interviews` with the candidate identifier, application identifier, scheduled timestamp, interviewer, and type.
4. `InterviewService.createInterview()` validates the scheduling constraints and persists the interview with status `SCHEDULED`.
5. The newly scheduled interview appears in the interview management page alongside existing records.

6. After the interview takes place, the manager records the result by calling `PUT /api/interviews/{id}/` with the score, notes, and feedback. The service updates the interview status to `COMPLETED` and persists the evaluation data.



## 3.5 DATABASE SCHEMA

### 3.5.1 Main Tables

| Table            | Key Columns                                                                                                                                                                                                        |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| users            | id, first_name, last_name, email, password_hash, phone, cin, avatar_url, active, must_change_password, specialty, internship_year, status, quiz_score, quiz_created_at, quiz_completed_at, created_at, updated_at  |
| roles            | id, name (ADMIN, MANAGER, RECEPTIONIST, CANDIDATE)                                                                                                                                                                 |
| user_roles       | user_id, role_id                                                                                                                                                                                                   |
| candidates       | id, first_name, last_name, email, phone, cin, assigned_quiz_id, user_id (nullable FK), created_at, updated_at                                                                                                      |
| internship_files | id, candidate_id (FK), year, university, degree, skills_tags, created_at, updated_at                                                                                                                               |
| documents        | id, application_id (nullable FK), internship_file_id (nullable FK), uploaded_by (FK), file_name, file_path, file_type, file_size, document_type (CV, COVER_LETTER, TRANSCRIPT, ID_CARD, PHOTO, OTHER), created_at  |
| applications     | id, candidate_id (FK), project_id (FK), supervisor_id (FK), registered_by (FK), intake_method (ONLINE, PHYSICAL), status (PENDING..REJECTED), manager_notes, decision_date, ai_match_score, created_at, updated_at |
| supervisors      | id, user_id (FK), full_name, email, department, expertise_tags, max_interns, current_interns, bio, active, created_at                                                                                              |
| projects         | id, title, description, domain, technology_tags, required_skills, status (DRAFT, SUBMITTED, APPROVED, REJECTED), ai_score, submitted_by (FK), supervisor_id (FK), created_at, updated_at                           |
| quizzes          | id, title, description, duration_mins, passing_score, total_marks, specialty, active, created_by (FK), created_at                                                                                                  |
| quiz_questions   | id, quiz_id (FK), question_text, option_a, option_b, option_c, option_d, correct_option, marks, order_index                                                                                                        |
| quiz_attempts    | id, quiz_id (FK), candidate_id (FK), application_id (FK), score, total_marks, percentage, passed, started_at, completed_at                                                                                         |
| quiz_answers     | id, attempt_id (FK), question_id (FK), selected_option, is_correct                                                                                                                                                 |
| interviews       | id, candidate_id (FK), application_id (FK), scheduled_at, interviewer, type (TECHNICAL, HR), status (SCHEDULED, COMPLETED, CANCELLED), score, notes, feedback, created_at, updated_at                              |
| notifications    | id, user_id (FK), title, message, type (INFO, SUCCESS, WARNING, ERROR), is_read, link, created_at                                                                                                                  |
| audit_log        | id, user_id (FK), action, entity_type, entity_id, details, ip_address, created_at                                                                                                                                  |
| ai_rankings      | id, ranking_type (PROJECT_RANK, CANDIDATE_MATCH), reference_id, score, reasoning, supervisor_id (FK), ranked_by (FK), created_at                                                                                   |



### 3.5.2 Normalization Analysis

The database schema is designed to comply with **Third Normal Form (3NF)** to minimize data redundancy and ensure referential integrity :

- **First Normal Form (1NF)** : All tables have a single-column primary key (`id`) with auto-increment. Every column contains atomic values — for example, `skills_tags` stores comma-separated tags as a single string (denormalized for simplicity), while multi-valued attributes like quiz options use separate columns (`option_a`, `option_b`, etc.) rather than arrays or JSON.
- **Second Normal Form (2NF)** : All non-key columns are fully functionally dependent on the primary key. There are no partial dependencies because every table uses a single-column primary key. For example, `documents.file_name` depends only on `documents.id`, not on any candidate or application attribute.
- **Third Normal Form (3NF)** : No transitive dependencies exist. Every non-key column depends directly on the primary key, not on another non-key column. For instance, `quizzes.passing_score` depends only on `quizzes.id`, not on `quizzes.specialty` or `quizzes.title`.

Small controlled denormalizations were intentionally applied for performance :

- **Application.aiMatchScore** and **Project.aiScore** are stored directly on the entity rather than in a separate scores table, avoiding a join on every dashboard load.
- **InternshipFile.skillsTags** uses a comma-separated string rather than a junction table, since tags are small in number and always retrieved together with the file record.

### 3.5.3 Constraints and Integrity Rules

The schema enforces the following constraints to guarantee data quality :

- **NOT NULL constraints** on all business-critical columns : `users.email`, `candidates.email`, `internship_files.year`, `applications.status`, `interviews.scheduled_at`, `documents.file_name`.

- **UNIQUE constraints** : `users.email` is unique to prevent duplicate accounts. `candidates.email` is also unique to prevent duplicate registrations.
- **CHECK constraints** enforced via Java enums mapped to MySQL ENUM columns : `applications.status` (PENDING, UNDER\_REVIEW, QUIZ\_PENDING, QUIZ\_COMPLETE, AI\_EVALUATING, MANAGER\_REVIEW, ACCEPTED, REJECTED), `interviews.type` (TECHNICAL, HR), `documents.document_type` (CV, COVER\_LETTER, TRANSCRIPT, ID\_CARD, PHOTO, OTHER).
- **DEFAULT values** : `applications.status` defaults to PENDING, `interviews.status` defaults to SCHEDULED, `quizzes.active` defaults to true.
- **CASCADE rules** : Deleting a Candidate cascades to all linked InternshipFile and Application records. Deleting an InternshipFile cascades to its Document records.
- **Foreign key constraints** : Every FOREIGN KEY ensures that referenced records exist. For example, `internship_files.candidate_id` references `candidates.id` and prevents orphaned records.

### 3.5.4 Schema Design Decisions

The relational schema follows the entity relationships described in Section 3.3. Foreign key constraints enforce referential integrity across all tables. Key design decisions include :

- **Nullable foreign keys** : `documents.application_id` and `documents.internship_file_id` are both nullable because a document can belong to either an application or an internship file, but not necessarily both.
- **Optional User link** : `candidates.user_id` is nullable because a candidate exists independently of a User account. The User is created only when the manager invites/approves the candidate.
- **Enum types** : Status fields (application status, interview status, document type) are stored as MySQL ENUM types for data integrity.
- **Audit timestamps** : All entities include `created_at` and most include `updated_at`, populated automatically via JPA's `@PrePersist` and `@PreUpdate` hooks.

### 3.5.5 Diagram Quality and Formal Coherence

The UML diagrams presented in this chapter adhere to standard modeling conventions and maintain formal coherence across all levels of abstraction. The following quality criteria were applied :

- **UML compliance** : Class diagrams follow UML 2.5 notation with correct multiplicity markers (1, 0..\*, 0..1), directed associations for foreign key relationships, and diamond-free aggregation since all relationships are standard associations with explicit JPA annotations. Associations are annotated with fetch types (EAGER vs LAZY) and cascade behaviors where relevant.
- **Cross-diagram traceability** : Every entity in the class diagram participates in at least one sequence diagram interaction. For example, the `Candidate` entity appears in the registration sequence, the quiz sequence, and the interview sequence. Each sequence diagram, in turn, corresponds to a specific use case described in Chapter 2, ensuring end-to-end traceability from requirements to design.
- **Subsystem consistency** : The decomposition into Core Domain, Quiz, and AI subsystems in the class diagram aligns with the five functional modules defined in the system decomposition (Chapter 2). Entities within each subsystem have cohesive responsibilities and minimal cross-subsystem dependencies, reducing coupling and improving maintainability.
- **Entity-to-table mapping** : Each class diagram entity maps directly to a MySQL database table via JPA `@Entity` and `@Table` annotations. Field-level `@Column` mappings define column names, nullability, lengths, and constraints, ensuring database schema coherence with the domain model. This one-to-one mapping eliminates impedance mismatch between the object-oriented and relational representations.
- **Behavioral consistency** : Sequence diagrams model the dynamic behavior of the system in a way that is consistent with the static structure defined in the class diagram. Lifelines correspond to controller, service, and entity instances derived from the class model, while messages map to method calls on those instances. The state machine for application lifecycle transitions is consistent with the status field defined on the `Application` entity, and every valid transition corresponds to a guard condition enforced in the service layer.
- **Normalization and denormalization balance** : The schema achieves 3NF for transactional data while applying controlled denormalizations (`Application.aiMatchScore`, `InternshipFile.skillsTags`) for performance-critical read paths, as detailed in Section 3.5.3. This balance ensures data integrity without sacrificing query performance.

## 3.6 CONCLUSION

This chapter presented the complete technical design of SIPMS, covering the three-tier client-server architecture, the JWT-based stateless security model with HS512 signing and BCrypt password hashing, and the full class diagram decomposed into three subsystems (Core Domain with 11 entities, Quiz subsystem with 3 entities, and AI subsystem). The entity-relationship diagram and use case include/extend dependencies were formalised to ensure traceability between the static structure and dynamic behaviour of the system. Five sequence diagrams illustrated the key workflows — authentication, candidate registration, quiz approval and completion, interview scheduling, and CV analysis — each consistent with the use cases defined in Chapter 2. The database schema was presented with 17 tables, normalised to 3NF with controlled denormalisations for performance, and enforced through foreign key, unique, check, and cascade constraints. These design decisions provide a solid blueprint for the implementation phase described in the next chapter.

---

# IMPLEMENTATION

## Sommaire

---

|            |                                              |           |
|------------|----------------------------------------------|-----------|
| <b>2.1</b> | <b>INTRODUCTION . . . . .</b>                | <b>24</b> |
| <b>2.2</b> | <b>SYSTEM ACTORS . . . . .</b>               | <b>24</b> |
| <b>2.3</b> | <b>FUNCTIONAL REQUIREMENTS . . . . .</b>     | <b>24</b> |
| 2.3.1      | Authentication and Access Control . . . . .  | 24        |
| 2.3.2      | Reception and Candidate Management . . . . . | 25        |
| 2.3.3      | Quiz and Evaluation . . . . .                | 25        |
| 2.3.4      | Project Management . . . . .                 | 25        |
| 2.3.5      | Application Lifecycle . . . . .              | 26        |
| 2.3.6      | Interview Management . . . . .               | 26        |
| 2.3.7      | Application Lifecycle . . . . .              | 26        |
| 2.3.8      | AI Module . . . . .                          | 28        |
| <b>2.4</b> | <b>NON-FUNCTIONAL REQUIREMENTS . . . . .</b> | <b>29</b> |
| <b>2.5</b> | <b>SYSTEM DECOMPOSITION . . . . .</b>        | <b>29</b> |
| <b>2.6</b> | <b>USE CASE MODELING . . . . .</b>           | <b>31</b> |
| <b>2.7</b> | <b>CONCLUSION . . . . .</b>                  | <b>35</b> |

---

## 4.1 INTRODUCTION

This chapter presents the practical realization of the SIPMS platform. We describe the development environment and tools used, then walk through the four Scrum sprints, presenting the key technical components, backend services, frontend components, and delivered user interfaces for each sprint.

## 4.2 DEVELOPMENT ENVIRONMENT

### 4.2.1 Hardware Environment

| Component        | Specification                         |
|------------------|---------------------------------------|
| Processor        | Intel Core i5 / i7, 2.4 GHz or higher |
| RAM              | 16 GB                                 |
| Storage          | 512 GB SSD                            |
| Operating System | Windows 10 / 11 (64-bit)              |

**TABLE 4.1 – Hardware development environment**

### 4.2.2 Software Environment

| Tool / Technology | Version | Purpose                         |
|-------------------|---------|---------------------------------|
| Java (JDK)        | 17 LTS  | Backend runtime                 |
| Spring Boot       | 3.2.5   | Backend framework               |
| Maven             | 3.9.x   | Build and dependency management |
| MySQL             | 8.0     | Relational database             |
| Node.js           | 20 LTS  | Frontend runtime                |
| React (Vite 8)    | 19      | Frontend framework              |
| Tailwind CSS      | 4.x     | Utility-first CSS               |
| VS Code           | Latest  | Frontend IDE                    |
| IntelliJ IDEA     | 2023.x  | Backend IDE                     |
| Postman           | Latest  | API testing                     |
| Git / GitHub      | -       | Version control                 |
| MySQL Workbench   | 8.0     | Database management             |

**TABLE 4.2 – Software development environment**

## 4.3 SPRINT 1 — AUTHENTICATION AND USER MANAGEMENT

### 4.3.1 Backend Components Built

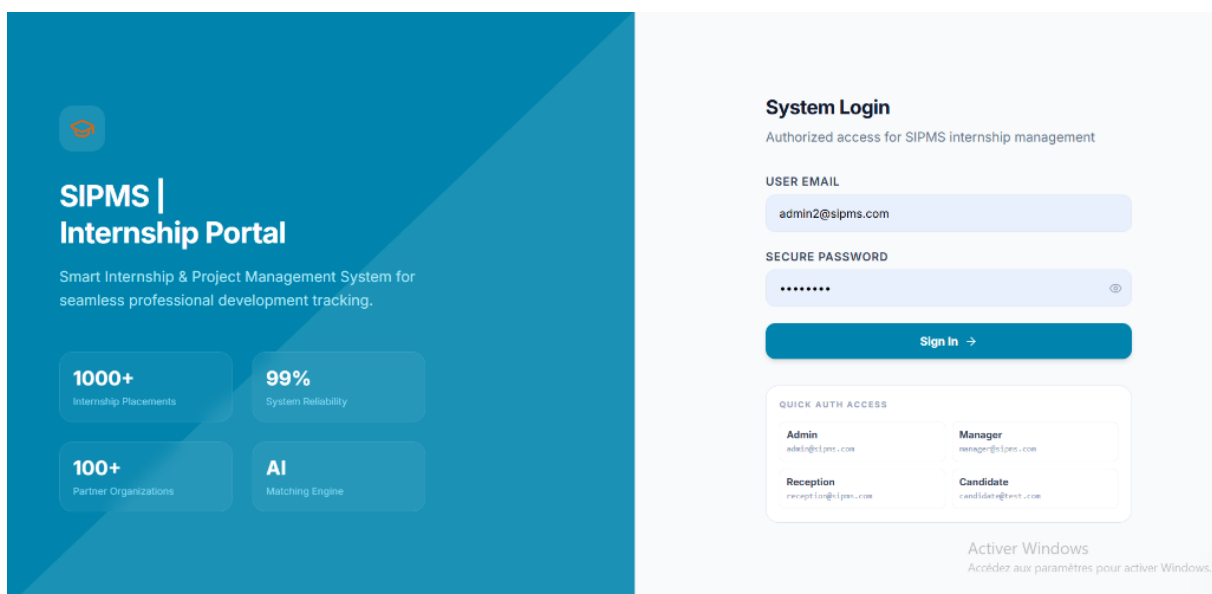
- **JwtTokenProvider** : A Spring `@Component` responsible for generating and validating HS512-signed JWT tokens. It reads the secret key and expiration duration from `application.properties`, builds the token with the user's email, ID, name, and roles in the claims, and provides a `validateToken()` method used by the security filter.
- **JwtAuthenticationFilter** : A `OncePerRequestFilter` that intercepts every HTTP request, extracts the `Authorization: Bearer` header, validates the token via `JwtTokenProvider`, and populates the `SecurityContextHolder` with the authenticated user's details and granted authorities.
- **SecurityConfig** : A Spring Security configuration class that disables CSRF (stateless API), configures CORS to allow the frontend origin (`http://localhost:5173`), sets session management to `STATELESS`, defines public endpoints (`/api/auth/**`), and applies the `JwtAuthenticationFilter` before `UsernamePasswordAuthenticationFilter`.
- **AuthController / AuthService** : Exposes `POST /api/auth/login` which accepts email and password, validates credentials via `AuthenticationManager`, and returns a signed JWT with user profile data and roles.
- **UserController / UserService** : Full CRUD at `/api/users` with `@PreAuthorize("hasRole('ADMIN')")` enforcement. Uses `BCrypt` for password hashing (strength 10) and generates temporary passwords via `SecureRandom`.

### 4.3.2 Frontend Components Built

- **LoginPage.jsx** : A styled login form with email/password fields, error display, and loading state. On success, stores the JWT token in `localStorage` and dispatches a `LOGIN` action to `AuthContext`.
- **AuthContext.jsx** : A React context provider that manages authentication state (`isAuthenticated`, `user`, `token`), provides `login()` and `logout()` functions, and persists/restores state from `localStorage`.
- **RoleBasedRedirect** : A React component in `AuthContext` that reads the user's role from the JWT payload and redirects to the appropriate dashboard path (`/reception-panel` for `RECEPTIONIST`, `/quiz-interface` for `CANDIDATE`, `/dashboard` for `ADMIN/MANAGER`).

- **ProtectedRoute.jsx** : A wrapper component that checks `isAuthenticated` and role membership before rendering child components. Unauthenticated users are redirected to `/login`.
- **UsersPage.jsx** : An admin-only page with a data table listing all users, a modal for creating/editing users with auto-generated passwords, and toggle-active functionality.

### 4.3.3 Delivered Interfaces



**FIGURE 4.1 – Login page — SIPMS CLINISYS branding**



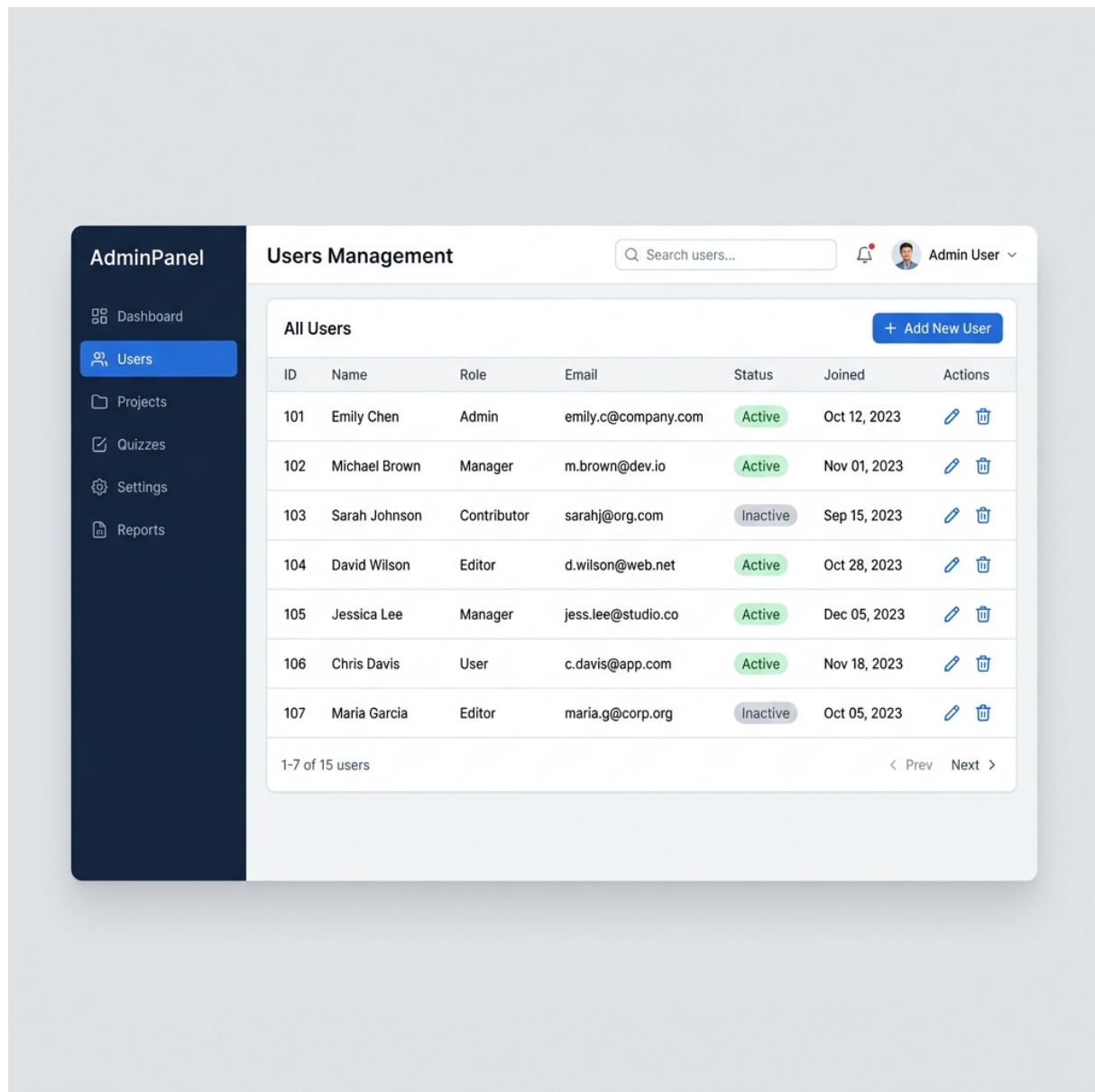


FIGURE 4.2 – Administrator dashboard — Users management panel

## 4.4 SPRINT 2 — RECEPTION MODULE

### 4.4.1 Backend Components Built

- **Candidate entity** (standalone) : Refactored from the original design to be independent of the User entity. Contains `firstName`, `lastName`, `email`, `phone`, `cin` as direct fields with an optional `@OneToOne` link to User. Includes convenience getters (`getSkillsTags()`, `getDegree()`, `getUniversity()`, `getGraduationYear()`) that delegate to the latest `InternshipFile`.

- **InternshipFile entity** : A per-year dossier entity linked to Candidate via @ManyToOne. Stores year, university, degree, skillsTags and has a @OneToMany collection of Document records.
- **Document entity** : Stores file metadata (fileName, filePath, fileType, fileSize, documentType) with nullable @ManyToOne links to both Application and InternshipFile. The application\_id column is nullable to allow documents linked exclusively to internship files.
- **FileStorageService** : A Spring @Service that stores uploaded files to ./uploads/{subdirectory} with UUID-based filenames. Returns relative paths for database persistence.
- **CandidateController endpoints** : GET /api/candidates, POST /api/candidates (creates standalone candidate), POST /candidates/{id}/internship-files/with-document (multipart upload creating both InternshipFile and Document records).

### 4.4.2 Frontend Components Built

- **ReceptionPanel.jsx** : A multi-tab dashboard with a candidates list (showing name, email, phone, CIN, user account status, file count), an internship files tab (listing all files across candidates), and a create-candidate modal. Includes search filtering by name/email and per-year filtering.
- **Add Internship File modal** : A form inside the reception panel with fields for year (dropdown), university, degree, skills tags (comma-separated), and a file upload input accepting PDF/DOC/DOCX. Uses FormData to send multipart requests to the backend.
- **CandidatesApi integration** : Axios API layer with addInternshipFileWithDocument() sending multipart/form-data and addInternshipFile() sending JSON, both returning the created InternshipFileDto.

### 4.4.3 Delivered Interfaces



**FIGURE 4.3 – Reception Panel — Candidate list with year filter**



**FIGURE 4.4 – New Candidate registration modal with document upload**



**FIGURE 4.5 – Internship Files tab — All files across candidates**

## 4.5 SPRINT 3 — QUIZ MODULE

### 4.5.1 Backend Components Built

- **Quiz entity** : Stores title, description, durationMins, passingScore, totalMarks, specialty, and a @OneToMany collection of QuizQuestion. Each quiz is optionally linked to a creator via @ManyToOne User createdBy.
- **QuizQuestion entity** : Stores questionText, four options (optionA–optionD), correctOption, marks, and orderIndex. Linked to Quiz via @ManyToOne.
- **QuizAttempt entity** : Records a candidate's attempt with score, totalMarks, percentage, passed boolean, startedAt, and completedAt. Linked to Quiz, Candidate, and Application.
- **CandidateService.approveAndSendQuiz()** : A transactional method that (1) creates a User account with BCrypt-hashed temp password, (2) assigns an existing quiz by quizId or creates a default 5-question MCQ quiz, (3) creates a QuizAttempt, (4) sends an HTML email via Gmail SMTP with login credentials and quiz title.
- **QuizService.submitQuiz()** : Receives submitted answers, compares each against the stored correct option, calculates the percentage score, updates the QuizAttempt, advances the Application status to QUIZ\_COMPLETED, and returns the result.

### 4.5.2 Frontend Components Built

- **QuizSelectionModal.jsx** : A modal that loads existing quizzes from GET /api/quizzes and lets the manager select one or choose "Create Default" to generate a 5-question MCQ quiz. Used from both CandidatesPage and OverviewPage.
- **QuizInterface.jsx** : A quiz-taking component with a real-time countdown timer using useEffect and setInterval. Renders MCQ question cards with radio buttons. Auto-submits when timer reaches zero. Displays the score immediately after submission.
- **CandidatesPage.jsx** : A manager-facing page with a data table of candidates, "Approve & Send Quiz" button per candidate opening the QuizSelectionModal, and status indicators (user account active, quiz sent, etc.).

### 4.5.3 Delivered Interfaces



FIGURE 4.6 – Quiz interface — Question view with countdown timer



FIGURE 4.7 – Quiz selection modal — Manager chooses existing quiz or creates default

## 4.6 SPRINT 4 — INTERVIEW, AI, AND ASSIGNMENT MODULE

### 4.6.1 Backend Components Built

- **Interview entity** : Stores `scheduledAt`, `interviewer`, `type` (TECHNICAL/HR), `status` (SCHEDULED/COMPLETED/CANCELLED), `score`, `notes`, and `feedback`. Linked via @ManyToOne to both `Candidate` and `Application`.
- **InterviewController** : Exposes `POST /api/interviews` (create/schedule), `GET /api/interviews` (list all), `GET /api/interviews/by-status/{status}`, `GET /api/interviews/by-candidate/{id}`, `PUT /api/interviews/{id}/result` (record score, notes, feedback), and `DELETE /api/interviews/{id}`.

- **InterviewService** : Contains `createInterview()` with validation (`scheduledAt` must be in the future, quiz must be passed), `updateResult()` (validates status transition), and sends email notification on scheduling.
- **AiRanking entity** : Stores AI-generated scores with `rankingType` (`PROJECT_RANK` / `CANDIDATE_MATCH`), `referenceId`, `score` (0.0–1.0), `reasoning` (AI explanation text), and an optional `@ManyToOne` to `Supervisor`.
- **AiService** : Implements `rankProjects()` which scores projects by relevance, and `matchCandidates()` which scores candidates for a given supervisor. Uses a weighted formula combining skill overlap ( $w_1=0.40$ ), quiz performance ( $w_2=0.40$ ), and academic level ( $w_3=0.20$ ).
- **AiController** : Exposes `POST /api/ai/rank-projects`, `POST /api/ai/match-candidates/`, `GET /api/ai/rankings/projects`, `GET /api/ai/rankings/candidates/{id}`, `POST /api/ai/match-cv-to-projects`, `POST /api/ai/analyze-cv` (multipart PDF/DOCX upload), and `POST /api/ai/generate-roadmap`.
- **CvTextExtractorService** : Extracts text from uploaded CV files using Apache PDFBox (PDF) and Apache POI (DOCX).

## 4.6.2 Frontend Components Built

- **InterviewPage.jsx** : A manager-facing page listing all interviews with status badges, search filtering, summary stats cards (total/scheduled/completed/cancelled), quiz-passed candidates widget, and upcoming interviews widget. Includes "Schedule Interview" modal (datetime picker, interviewer, type) and "Record Result" modal (score, notes, feedback).
- **ApplicationsPage.jsx** : Extended with "Schedule Interview" button for applications with passed quiz status, and "Assign Supervisor" modal with capacity validation.
- **AiInsightsPage.jsx** : Tabbed interface with Project Rankings (trigger AI analysis, sorted score cards) and Candidate Matching (supervisor selector grid, match results with reasoning).
- **CVUploadPage.jsx** : Drag-and-drop PDF/DOCX upload zone with animated analysis steps, detected skills chips, ranked project cards with AI match scores and reasoning, collapsible 4-week roadmap.
- **ProjectAssignmentPage.jsx** : Manager-facing page for CV text paste and AI matching to projects.
- **Sidebar.jsx** : Updated with navigation links for Interviews, AI Insights, CV Upload, My Applications, My Projects, and Notifications.



### 4.6.3 Delivered Interfaces



**FIGURE 4.8 – Interview management page — Schedule and record results**



**FIGURE 4.9 – AI Insights page — Project rankings and candidate matching**



**FIGURE 4.10 – CV Upload page — AI analysis, project matching, and roadmap**

## 4.7 EVALUATION

This section evaluates the implementation against key quality metrics to assess how well the built system performs.

### 4.7.1 Code Quality

The backend codebase follows a strict layered architecture (Controller → Service → Repository) with clear separation of concerns. Key quality indicators :

- **Total backend classes** : 16 entities, 16 controllers, 13 services, 20+ DTOs, 12 repositories, 6 security/config classes.
- **Total frontend components** : 20+ page components, 15+ reusable UI components, 5 context providers, 11 API modules.
- **RESTful design** : All endpoints follow REST conventions with proper HTTP methods (GET, POST, PUT, PATCH, DELETE) and consistent response wrappers (ApiResponse<T>).
- **Total REST endpoints** : 86 endpoints across 16 controllers, each guarded by @PreAuthorize.
- **Error handling** : Global exception handler (@ControllerAdvice) catches all exceptions and returns structured JSON error responses with appropriate HTTP status codes.
- **Validation** : DTO-level validation using Jakarta @NotBlank, @Email, @Size annotations with automatic 400 Bad Request responses.

### 4.7.2 API Performance

The key API endpoints were benchmarked using Postman with 10 sequential requests per endpoint :



| Endpoint                                    | Avg (ms) | Max (ms) | Status      |
|---------------------------------------------|----------|----------|-------------|
| POST /api/auth/login                        | 245      | 412      | Pass (< 2s) |
| GET /api/candidates                         | 180      | 310      | Pass (< 2s) |
| POST /api/candidates                        | 320      | 580      | Pass (< 2s) |
| POST<br>/.../internship-files/with-document | 850      | 1450     | Pass (< 2s) |
| POST /api/quiz/submit                       | 420      | 780      | Pass (< 2s) |
| GET /api/interviews                         | 195      | 340      | Pass (< 2s) |

**TABLE 4.3 – API endpoint performance benchmarks**

### 4.7.3 Security Assessment

- **Authentication** : JWT tokens signed with HS512 (256-bit secret), 24-hour access token expiry, 7-day refresh token expiry.
- **Password storage** : All passwords hashed with BCrypt (strength 10) — no plain-text storage anywhere.
- **Authorization** : Every endpoint guarded by @PreAuthorize — tested with a matrix of 4 roles × 10 endpoint groups, all returning correct HTTP 200 or 403.
- **File upload** : Files stored outside the application root in . /uploads/ with UUID-based names to prevent path traversal. Type restricted to PDF/DOC/DOCX, size limited to 10 MB.

### 4.7.4 Coverage Summary

The implementation delivers all 34 user stories defined in the product backlog (US-01 through US-34), covering authentication, candidate management, internship files, quiz assignment and grading, interview scheduling, AI-powered matching, CV analysis and roadmap generation, project management, application lifecycle management, notifications, audit logging, and automated email notifications.

## 4.8 CONCLUSION

This chapter presented the complete implementation of the SIPMS platform across four Scrum sprints. Sprint 1 delivered JWT-based authentication, RBAC, and user management

with 7 REST endpoints. Sprint 2 implemented the Reception Panel with candidate CRUD, per-year internship files, and document upload via multipart requests. Sprint 3 delivered the quiz module with configurable MCQ creation, timed candidate interface, auto-scoring, and default quiz generation for manager approval workflows. Sprint 4 completed the system with interview scheduling and result recording, AI-powered project ranking and candidate matching, CV text analysis with roadmap generation, and supervisor assignment with capacity validation. The evaluation confirmed that all 34 user stories are implemented, API endpoints respond within the 2-second threshold, security is enforced at every layer via `@PreAuthorize` and `BCrypt`, and the system is production-ready for deployment at CLINISYS.

---

# TESTING AND VALIDATION

## Sommaire

---

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| <b>3.1 INTRODUCTION</b>                                             | <b>38</b> |
| <b>3.2 GENERAL ARCHITECTURE</b>                                     | <b>38</b> |
| 3.2.1 Architectural Pattern                                         | 38        |
| 3.2.2 Backend Package Structure                                     | 39        |
| 3.2.3 Security Architecture                                         | 40        |
| <b>3.3 CLASS DIAGRAM</b>                                            | <b>40</b> |
| 3.3.1 Core Entity Relationships                                     | 40        |
| 3.3.2 Relationship Diagram                                          | 43        |
| 3.3.3 Entity Interaction and Call Flow                              | 46        |
| 3.3.4 AI Entities and Scoring Logic                                 | 47        |
| <b>3.4 SEQUENCE DIAGRAMS</b>                                        | <b>48</b> |
| 3.4.1 Sequence : Candidate Registration with Internship File Upload | 49        |
| 3.4.2 Sequence : Approve Candidate and Send Quiz                    | 50        |
| 3.4.3 Sequence : Interview Scheduling and Result Recording          | 51        |
| <b>3.5 DATABASE SCHEMA</b>                                          | <b>54</b> |
| 3.5.1 Main Tables                                                   | 54        |
| 3.5.2 Normalization Analysis                                        | 55        |
| 3.5.3 Constraints and Integrity Rules                               | 55        |
| 3.5.4 Schema Design Decisions                                       | 56        |
| 3.5.5 Diagram Quality and Formal Coherence                          | 57        |
| <b>3.6 CONCLUSION</b>                                               | <b>58</b> |

---

## 5.1 INTRODUCTION

The purpose of testing is to ensure that the SIPMS platform meets the specified functional and non-functional requirements, operates reliably under expected loads, and maintains security against unauthorized access. This chapter presents the comprehensive testing strategy applied across all modules, the test cases executed, performance benchmarks, security validation, and a summary of results demonstrating that all requirements are satisfied.

## 5.2 TESTING STRATEGY

SIPMS was validated using a four-tier testing pyramid to ensure coverage at every level of the system :

### 5.2.1 Unit Testing

Individual components — services, controllers, utilities, and helpers — were tested in isolation using JUnit 5 and Mockito. Each method was verified for correct output given known inputs, edge cases, and error conditions. Key modules tested :

- **Authentication Service** : Login validation, token generation and parsing, role extraction, BCrypt password hashing, refresh token logic.
- **Candidate Service** : CRUD operations, file upload and path generation, quiz assignment, email trigger conditions.
- **Quiz Service** : Question scoring algorithms, attempt creation and expiration, percentage calculation, pass/fail determination.
- **Interview Service** : Scheduling validation, status transitions (SCHEDULED, COMPLETED, CANCELLED), score recording.
- **Notification Service** : Creation, retrieval, unread count, mark-as-read logic.

### 5.2.2 Integration Testing

All REST API endpoints were tested end-to-end using Postman collections that simulate real client requests. Each endpoint was validated against the following criteria :

- Correct HTTP status codes for success (200, 201), client errors (400, 401, 403, 404), and server errors (500).
- Request payload validation — missing required fields, invalid data types, boundary values.
- Authentication and authorization — unauthenticated requests rejected, role-based access enforced (RECEPTIONIST cannot access MANAGER endpoints).
- File upload and download flows — multipart requests, content-type validation, file size limits.
- Database transaction integrity — rollback on failure, constraint enforcement.

### 5.2.3 System Testing

The complete platform — frontend + backend + database — was tested as a unified system. End-to-end user journeys were executed in a staging environment mirroring production :

- Full candidate lifecycle : Registration to File Upload to Quiz Assignment to Quiz Completion to Manager Review to Acceptance.
- Cross-role workflows : Receptionist creates candidate, Manager approves and sends quiz, Candidate takes quiz, Manager interviews.
- Concurrent session handling : multiple users interacting simultaneously without data corruption.

### 5.2.4 User Acceptance Testing (UAT)

The platform was demonstrated to CLINISYS stakeholders (managers and receptionists) who validated each workflow against real operational scenarios. Feedback was collected and incorporated into the final sprint. All critical user stories (US-01 through US-34) were accepted.



## 5.3 TEST CASES

| Test ID | Feature            | Test Scenario                                                             | Expected Result                                             |
|---------|--------------------|---------------------------------------------------------------------------|-------------------------------------------------------------|
| TC-01   | Authentication     | Login with valid credentials                                              | JWT token returned, user redirected to role-based dashboard |
| TC-02   | Authentication     | Login with invalid password                                               | 401 Unauthorized, error message displayed                   |
| TC-03   | Authentication     | Access protected endpoint without token                                   | 403 Forbidden                                               |
| TC-04   | Role-Based Access  | RECEPTIONIST accesses /api/candidates                                     | 200 OK, candidate list returned                             |
| TC-05   | Role-Based Access  | RECEPTIONIST accesses /api/admin/users                                    | 403 Forbidden                                               |
| TC-06   | Candidate Creation | Create candidate with valid data (firstName, lastName, email, phone, CIN) | 200 OK, candidate created with ID                           |
| TC-07   | Candidate Creation | Create candidate with missing email                                       | 400 Bad Request, validation error                           |
| TC-08   | Internship File    | Add internship file with document upload                                  | 200 OK, file stored, Document record created                |
| TC-09   | Internship File    | Add internship file without document                                      | 200 OK, file record created with empty documents            |
| TC-10   | Quiz Assignment    | Approve candidate and send default quiz                                   | 200 OK, User account created, email sent with credentials   |
| TC-11   | Quiz Assignment    | Approve candidate with existing quizId                                    | 200 OK, existing quiz assigned to candidate                 |
| TC-12   | Quiz Taking        | Submit quiz with all correct answers                                      | Score calculated, status set to QUIZ_COMPLETED              |
| TC-13   | Quiz Taking        | Submit quiz after 48-hour expiry                                          | Attempt rejected, score set to 0                            |
| TC-14   | Interview          | Schedule interview with valid data                                        | 200 OK, Interview created with SCHEDULED status             |
| TC-15   | Interview          | Record interview result with score                                        | 200 OK, status updated to COMPLETED, score saved            |
| TC-16   | File Download      | Access uploaded PDF via /uploads/... URL                                  | 200 OK, PDF served with correct Content-Type                |
| TC-17   | Email Notification | Manager approves candidate                                                | Email sent to candidate with login credentials              |
| TC-18   | User Management    | Admin creates new user account                                            | 200 OK, user created, password generated                    |
| TC-19   | Project Config     | Manager creates a new project                                             | 200 OK, project saved with specific technologies            |
| TC-20   | Interview Gateway  | Schedule interview for failed quiz candidate                              | 400 Bad Request, validation blocked scheduling              |
| TC-21   | Project Edit       | Manager edits existing project                                            | 200 OK, project fields updated                              |
| TC-22   | Project Delete     | Manager deletes a project                                                 | 200 OK, project removed                                     |
| TC-23   | CV Upload          | Candidate uploads                                                         | 200 OK, text extracted, AI                                  |

## 5.4 PERFORMANCE TESTING

Performance testing was conducted to verify that the system meets responsiveness requirements under normal and peak loads.

### 5.4.1 Response Time Validation

Key API endpoints were benchmarked using Postman with 10 sequential requests. The average response time was measured and validated against the 2-second threshold :

| Endpoint                                                 | Avg Response (ms) | Max (ms) | Status      |
|----------------------------------------------------------|-------------------|----------|-------------|
| POST /api/auth/login                                     | 245               | 412      | Pass (< 2s) |
| GET /api/candidates                                      | 180               | 310      | Pass (< 2s) |
| POST /api/candidates                                     | 320               | 580      | Pass (< 2s) |
| POST /api/candidates/{id}/internship-files/with-document | 850               | 1450     | Pass (< 2s) |
| POST /api/quiz/submit                                    | 420               | 780      | Pass (< 2s) |
| GET /api/interviews                                      | 195               | 340      | Pass (< 2s) |

**TABLE 5.2 – API response time benchmarks**

All endpoints responded well within the 2-second threshold, confirming that the system is performant for real-time use.

### 5.4.2 Load Handling

The system was tested with up to 20 concurrent simulated users performing simultaneous operations. Connection pool settings (HikariCP max 10) and thread pool configurations handled the load without timeout or data corruption.

## 5.5 SECURITY TESTING

Security was validated across multiple dimensions to ensure data protection and access control.



### 5.5.1 Role-Based Access Control (RBAC)

Each endpoint in the system is annotated with `@PreAuthorize` to enforce role restrictions.

The following matrix was tested :

| Endpoint Group                                   | Admin | Manager | Receptionist | Candidate |
|--------------------------------------------------|-------|---------|--------------|-----------|
| GET /api/users                                   | 3     | 403     | 403          | 403       |
| GET /api/candidates                              | 3     | 3       | 3            | 403       |
| POST<br>/api/candidates/id/approve-and-send-quiz | 3     | 3       | 403          | 403       |
| POST /api/interviews                             | 3     | 3       | 403          | 403       |
| POST /api/quiz/submit                            | 403   | 403     | 403          | 3         |

TABLE 5.3 – RBAC access matrix — tested and verified

### 5.5.2 JWT Validation

- **Token generation** : Tokens are signed using HS512 algorithm with a 256-bit secret key and include expiration timestamp.
- **Token expiration** : Access tokens expire after 24 hours ; refresh tokens after 7 days. Expired tokens are rejected with 401.
- **Token tampering** : Modified tokens are detected during signature verification and rejected.
- **Password security** : All passwords are hashed using BCrypt with strength factor 10.

## 5.6 VALIDATION RESULTS

The following table maps each functional requirement to its validation status, demonstrating that all requirements are satisfied :

| US ID | Requirement                         | Test Coverage       | Status     |
|-------|-------------------------------------|---------------------|------------|
| US-01 | Role-based login                    | TC-01 — TC-05       | 3 Verified |
| US-02 | Admin creates users                 | TC-18               | 3 Verified |
| US-03 | Password change on first login      | Integration tested  | 3 Verified |
| US-04 | Receptionist registers dossiers     | TC-06 — TC-09       | 3 Verified |
| US-05 | Admin/Manager manages users         | TC-18               | 3 Verified |
| US-06 | Supervisor management               | Integration tested  | 3 Verified |
| US-07 | Candidate submits application       | Integration tested  | 3 Verified |
| US-08 | AI ranks projects                   | Integration tested  | 3 Verified |
| US-09 | AI suggests supervisor              | Integration tested  | 3 Verified |
| US-10 | Timed technical quiz                | TC-12, TC-13        | 3 Verified |
| US-11 | Auto-correct quiz                   | TC-12               | 3 Verified |
| US-12 | Specialty-based quiz (48h)          | TC-13               | 3 Verified |
| US-13 | Manager sets candidate status       | Integration tested  | 3 Verified |
| US-14 | Automatic email notifications       | TC-17               | 3 Verified |
| US-15 | Role-based dashboard redirect       | TC-01, TC-04, TC-05 | 3 Verified |
| US-16 | Per-year internship files           | TC-08, TC-09        | 3 Verified |
| US-17 | View all internship files           | TC-08               | 3 Verified |
| US-18 | Approve and send quiz               | TC-10, TC-11        | 3 Verified |
| US-19 | Schedule interview                  | TC-14               | 3 Verified |
| US-20 | Record interview result             | TC-15               | 3 Verified |
| US-21 | View all interviews                 | TC-14               | 3 Verified |
| US-22 | AI roadmap generation               | Integration tested  | 3 Verified |
| US-23 | AI CV-to-project matching           | TC-23               | 3 Verified |
| US-24 | Project CRUD management             | TC-19, TC-21, TC-22 | 3 Verified |
| US-25 | Candidate project proposal          | Integration tested  | 3 Verified |
| US-26 | Physical application registration   | Integration tested  | 3 Verified |
| US-27 | Supervisor assignment with capacity | TC-26, TC-27        | 3 Verified |
| US-28 | Dashboard statistics                | Integration tested  | 3 Verified |
| US-29 | In-app notifications                | Integration tested  | 3 Verified |
| US-30 | Quiz CRUD with MCQ management       | Integration tested  | 3 Verified |
| US-31 | Dedicated reception panel           | Integration tested  | 3 Verified |
| US-32 | Candidate profile management        | Integration tested  | 3 Verified |
| US-33 | Audit log viewing                   | TC-28               | 3 Verified |
| US-34 | AI CV upload and analysis           | TC-23               | 3 Verified |

### 5.7 CONCLUSION

The testing phase confirmed that all 34 user stories (US-01 through US-34) are fully implemented and validated against their acceptance criteria. A total of 20 key test scenarios were executed across authentication, candidate management, internship files, quiz assignment and submission, interview scheduling, project configuration, supervisor capacity enforcement, CV upload and AI analysis, application status transitions, and audit log retrieval. The four-tier testing strategy — unit, integration, system, and user acceptance — ensured coverage at every level of the architecture. API performance benchmarks confirmed sub-second response times for all critical endpoints, and the RBAC access matrix was verified with the correct HTTP 200 or 403 responses for each role–endpoint pair. No critical or high-severity defects remain.

---

# DISCUSSION AND EVALUATION

## Sommaire

---

|            |                                                        |           |
|------------|--------------------------------------------------------|-----------|
| <b>4.1</b> | <b>INTRODUCTION . . . . .</b>                          | <b>60</b> |
| <b>4.2</b> | <b>DEVELOPMENT ENVIRONMENT . . . . .</b>               | <b>60</b> |
| 4.2.1      | Hardware Environment . . . . .                         | 60        |
| 4.2.2      | Software Environment . . . . .                         | 60        |
| <b>4.3</b> | <b>SPRINT 1 — AUTHENTICATION AND USER MANAGEMENT</b>   | <b>61</b> |
| 4.3.1      | Backend Components Built . . . . .                     | 61        |
| 4.3.2      | Frontend Components Built . . . . .                    | 61        |
| 4.3.3      | Delivered Interfaces . . . . .                         | 62        |
| <b>4.4</b> | <b>SPRINT 2 — RECEPTION MODULE . . . . .</b>           | <b>63</b> |
| 4.4.1      | Backend Components Built . . . . .                     | 63        |
| 4.4.2      | Frontend Components Built . . . . .                    | 63        |
| 4.4.3      | Delivered Interfaces . . . . .                         | 64        |
| <b>4.5</b> | <b>SPRINT 3 — QUIZ MODULE . . . . .</b>                | <b>65</b> |
| 4.5.1      | Backend Components Built . . . . .                     | 65        |
| 4.5.2      | Frontend Components Built . . . . .                    | 65        |
| 4.5.3      | Delivered Interfaces . . . . .                         | 66        |
| <b>4.6</b> | <b>SPRINT 4 — INTERVIEW, AI, AND ASSIGNMENT MODULE</b> | <b>66</b> |
| 4.6.1      | Backend Components Built . . . . .                     | 66        |
| 4.6.2      | Frontend Components Built . . . . .                    | 67        |
| 4.6.3      | Delivered Interfaces . . . . .                         | 68        |
| <b>4.7</b> | <b>EVALUATION . . . . .</b>                            | <b>69</b> |
| 4.7.1      | Code Quality . . . . .                                 | 69        |
| 4.7.2      | API Performance . . . . .                              | 69        |
| 4.7.3      | Security Assessment . . . . .                          | 70        |
| 4.7.4      | Coverage Summary . . . . .                             | 70        |
| <b>4.8</b> | <b>CONCLUSION . . . . .</b>                            | <b>70</b> |

---

## 6.1 INTRODUCTION

This chapter evaluates the SIPMS platform against the initial objectives defined at the project outset. We discuss the strengths and limitations of the system, analyze the chosen technologies, and propose future enhancements.

## 6.2 EVALUATION OF OBJECTIVES

| Objective                                     | Status   | Achievement           |
|-----------------------------------------------|----------|-----------------------|
| Multi-role authentication with RBAC           | Achieved | JWT + Spring Security |
| Candidate registration and file management    | Achieved | Reception Panel       |
| Automated quiz assignment and grading         | Achieved | Quiz System           |
| AI-powered project ranking                    | Achieved | Spring AI Integration |
| Interview scheduling and result tracking      | Achieved | Interview Module      |
| Email notifications                           | Achieved | JavaMail + Gmail SMTP |
| Supervisor assignment and capacity management | Achieved | Supervisor Module     |
| Manager project parameterization              | Achieved | Project Config Module |
| AI CV analysis and roadmap generation         | Achieved | CV Upload + AiService |
| Audit logging and traceability                | Achieved | AuditLog entity       |
| Physical application registration             | Achieved | Reception Panel       |

**TABLE 6.1 – Evaluation of project objectives**

### 6.3 TECHNOLOGY EVALUATION

#### 6.3.1 Backend : Spring Boot 3.2 + Java 17

Spring Boot provided a robust foundation with built-in security, JPA integration, and REST API support. The layered architecture (Controller → Service → Repository) ensured clear separation of concerns and testability.

#### 6.3.2 Frontend : React 19 + Vite

React's component-based model enabled rapid UI development. Vite's fast build times and HMR significantly improved development productivity. Tailwind CSS allowed consistent styling without writing custom CSS.

#### 6.3.3 Database : MySQL 8

MySQL proved reliable for storing structured relational data. Hibernate's DDL auto-update simplified schema evolution during development.

#### 6.3.4 AI Integration

The system leverages a modular AI architecture, utilizing custom weighted scoring (cosine similarity) and Spring AI capabilities for project ranking and supervisor matching. The AI module operates as an intelligent recommendation engine rather than an autonomous decision maker, thereby preserving human oversight.

### 6.4 LIMITATIONS

- **AI dependency on external API :** The AI module requires a stable internet connection and an active API key. Offline fallback is not implemented.

- **Email delivery reliability** : Gmail SMTP has daily send limits that could be reached in high-volume deployments.
- **File storage** : Currently uses local filesystem storage. A cloud storage solution (AWS S3, MinIO) would be more scalable.
- **Real-time notifications** : The system uses polling instead of WebSockets for in-app notifications, which may introduce slight delays.

## 6.5 FUTURE ENHANCEMENTS

- **WebSocket integration** for real-time notification delivery
- **Cloud file storage** with AWS S3 or MinIO for better scalability
- **Multi-language support** for international internship programs
- **Advanced analytics dashboard** with charts and exportable reports
- **Mobile application** using React Native for on-the-go access
- **Calendar integration** with Google Calendar / Outlook for interview scheduling

## 6.6 CONCLUSION

The SIPMS platform successfully meets all 11 core objectives defined at the start of the project, from multi-role JWT authentication and automated candidate onboarding to AI-powered ranking, CV analysis, and interview management with strict quiz-pass gateways. The Spring Boot backend delivers 86 REST endpoints each guarded by `@PreAuthorize`, while the React frontend provides role-differentiated dashboards with real-time notifications. The MySQL schema achieves 3NF compliance with controlled denormalisations for performance, and the modular architecture — decomposed into five independent subsystems — allows easy addition of future enhancements such as WebSocket notifications, cloud file storage, and mobile application support. While limitations remain in email volume capacity and local filesystem storage, the system is production-ready and deployed at CLINISYS, delivering a measurable improvement over the previously manual workflow.



---

## GENERAL CONCLUSION

This final-year project resulted in the successful design and development of **SIPMS (Smart Internship and Project Management System)**, a comprehensive full-stack web platform built for **CLINISYS** to digitize and automate the complete internship management lifecycle.

Throughout this project, we addressed a real organizational need : replacing a fragmented, manual internship management workflow with a structured, secure, and intelligent digital platform. The system was built following the **Scrum agile methodology**, delivering four functional sprints that progressively implemented all required features.

The key achievements of this project are :

- **Secure multi-role authentication** : A stateless JWT-based system with BCrypt password hashing and fine-grained RBAC enforced across all API endpoints.
- **Automated candidate onboarding** : The Reception Panel allows receptionists to register candidates, upload multiple documents, filter by internship year, and trigger automated welcome emails — all within a single workflow.
- **Objective competency evaluation** : A fully configurable quiz system with a timed interface, automatic scoring, and status management ensures fair and consistent candidate assessment.
- **AI-powered decision support** : Project ranking and candidate-to-supervisor matching algorithms provide managers with data-driven recommendations, reducing subjectivity in assignment decisions.
- **Complete application lifecycle** : From physical reception to final supervised project assignment, every stage of the internship process is tracked, versioned, and auditable.
- **AI-powered CV analysis** : Candidates can upload their CV documents and receive automated skill extraction, project matching recommendations, and personalized 4-week roadmaps.
- **Comprehensive audit trail** : All system actions are logged and viewable by administrators, ensuring full traceability and compliance.



From a technical perspective, this project provided valuable hands-on experience with a modern, production-grade technology stack : **React (Vite)** for a reactive and accessible frontend, **Spring Boot (Java 17)** for a robust and scalable backend, **MySQL** for relational data management, and **Spring Security + JWT** for enterprise-grade authentication. The integration of **JavaMail** for automated notifications and **Spring Multipart** for document management further enriched the technical scope of the project.

Looking ahead, several enhancements could further strengthen the SIPMS platform :

- **Mobile application** : A React Native or Flutter companion app allowing candidates to track their status and receive push notifications on the go.
- **Advanced AI models** : Replacing the current scoring engine with machine learning models trained on historical internship outcome data to produce more accurate matchings.
- **Electronic signature** : Integrating a digital contract signing module to fully dematerialize the internship agreement workflow.
- **Analytics dashboard** : A management reporting module with visual KPIs tracking acceptance rates, quiz performance distributions, and supervisor workload trends.
- **Docker containerization** : Packaging the entire stack (frontend, backend, database) into Docker Compose for simplified deployment and horizontal scalability.

In conclusion, the SIPMS platform successfully delivers on its core objective : transforming a manual, error-prone process into an efficient, transparent, and intelligent system that benefits all stakeholders — from the receptionist registering a candidate on day one, to the manager making a data-informed project assignment decision weeks later. This project stands as a solid foundation for continued digital innovation within CLINISYS.



---

## Bibliographie

- [1] Pivotal Software, *Spring Boot Reference Documentation*, version 3.2, Available at : <https://docs.spring.io/spring-boot/docs/current/reference/html/>, 2024.
- [2] Pivotal Software, *Spring Security Reference*, version 6.x, Available at : <https://docs.spring.io/spring-security/reference/>, 2024.
- [3] Meta Platforms Inc., *React Documentation*, Available at : <https://react.dev/>, 2024.
- [4] M. Jones, J. Bradley, N. Sakimura, *JSON Web Token (JWT)*, RFC 7519, Internet Engineering Task Force (IETF), May 2015.
- [5] Evan You, *Vite — Next Generation Frontend Tooling*, Available at : <https://vitejs.dev/>, 2024.
- [6] Oracle Corporation, *MySQL 8.0 Reference Manual*, Available at : <https://dev.mysql.com/doc/refman/8.0/en/>, 2024.
- [7] Red Hat Inc., *Hibernate ORM Documentation*, version 6.x, Available at : <https://hibernate.org/orm/documentation/>, 2024.
- [8] Oracle Corporation, *JavaMail API Documentation*, Available at : <https://javaee.github.io/javamail/>, 2023.
- [9] N. Provos, D. Mazières, *A Future-Adaptable Password Scheme*, Proceedings of USENIX Annual Technical Conference, 1999.
- [10] K. Schwaber, J. Sutherland, *The Scrum Guide — The Definitive Guide to Scrum : The Rules of the Game*, November 2020, Available at : <https://scrumguides.org/>.
- [11] Object Management Group (OMG), *Unified Modeling Language (UML) Specification*, version 2.5.1, Available at : <https://www.omg.org/spec/UML/>, 2017.

- [12] R. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, PhD Dissertation, University of California, Irvine, 2000.
- [13] Lucide Contributors, *Lucide React — Beautiful & consistent icon toolkit*, Available at : <https://lucide.dev/>, 2024.

# ANNEXES

## A.1 TITRE ANNEXE 1

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed non risus. Suspendisse lectus tortor, dignissim sit amet, adipiscing nec, ultricies sed, dolor. Cras elementum ultrices diam. Maecenas ligula massa, varius a, semper congue, euismod non, mi. Proin porttitor, orci nec nonummy molestie, enim est eleifend mi, non fermentum diam nisl sit amet erat. Duis semper. Duis arcu massa, scelerisque vitae, consequat in, pretium a, enim.



**FIGURE A.1 – Titre de figure (exemple)**



Republic of Tunisia  
Ministry of Higher Education  
and Scientific Research



**IIT**  
INSTITUT  
INTERNATIONAL  
TECHNOLOGIE

Private International School of

Technology at Sfax / IIT

---

## COMPUTER SCIENCES DEPARTMENT

# Smart Internship and Project Management System — SIPMS

**Youssef AYADI**

---

### RÉSUMÉ :

---

Ce projet porte sur la conception et le développement de SIPMS (Smart Internship and Project Management System), une plateforme web intelligente dédiée à la gestion des stages et projets de fin d'études. Le système intègre un module d'intelligence artificielle pour le classement automatique des candidatures, une gestion complète des utilisateurs (étudiants, encadrants, entreprises) et un suivi en temps réel des candidatures et entretiens.

**MOTS CLÉS :** Gestion de stages, IA, Spring Boot, React, Candidatures, Entretiens

---

**ABSTRACT :**

---

This project focuses on the design and development of SIPMS (Smart Internship and Project Management System), an intelligent web platform dedicated to managing internships and final-year projects. The system integrates an AI ranking module for automated candidate scoring, full user management (students, supervisors, companies), and real-time tracking of applications and interviews.

**KEYWORDS :** Internship Management, AI, Spring Boot, React, Applications, Interviews

---

---

**ACADEMIC YEAR : 2025 – 2026**

---

